

# Ontwikkeling van een smartphone sdk voor het obfusceren van locatie data

**Milan SCHOLLIER**

Promotor(en): prof. dr. Vincent Naessen Masterproef ingediend tot het behalen van  
de graad van master of Science in de  
Co-promotor(en): dr. ing. Michiel Willocx industriële wetenschappen: Master in de  
industriële wetenschappen elektronica-ICT:  
smart applications

Academiejaar 2024 - 2025



©Copyright KU Leuven

Deze masterproef is een examendocument dat niet werd gecorrigeerd voor eventuele vastgestelde fouten.

Zonder voorafgaande schriftelijke toestemming van zowel de promotor(en) als de auteur(s) is overnemen, kopiëren, gebruiken of realiseren van deze uitgave of gedeelten ervan verboden. Voor aanvragen i.v.m. het overnemen en/of gebruik en/of realisatie van gedeelten uit deze publicatie, kan u zich richten tot KU Leuven Technologicampus Gent, Gebroeders De Smetstraat 1, B-9000 Gent, +32 92 65 86 10 of via e-mail [iiw.gent@kuleuven.be](mailto:iiw.gent@kuleuven.be).

Voorafgaande schriftelijke toestemming van de promotor(en) is eveneens vereist voor het aanwenden van de in deze masterproef beschreven (originele) methoden, producten, schakelingen en programma's voor industrieel of commercieel nut en voor de inzending van deze publicatie ter deelname aan wetenschappelijke prijzen of wedstrijden.





# Dankwoord

Voor u ligt de thesis "Ontwikkeling van een smartphone sdk voor het obfusceren van locatie data". Deze thesis is geschreven om te voldoen aan de afstudeereisen van de Master in de industriële wetenschappen elektronica-ICT: smart applications aan de Katholieke Universiteit van Leuven.

Ervaringen:

Graag maak ik van deze gelegenheid gebruik om enkele mensen te bedanken die een belangrijke rol hebben gespeeld bij het tot stand komen van deze thesis.

In de eerste plaats wil ik mijn promotor, prof. dr. Vincent Naessens, bedanken voor het vertrouwen in dit onderzoek en voor het aanreiken van dit boeiende onderwerp. Mijn oprechte dank gaat ook uit naar mijn masterproefbegeleider, dr. ing. Michiel Willocx, voor de deskundige begeleiding, waardevolle feedback en voortdurende motivatie gedurende het hele traject. Daarnaast wil ik de medewerkers van Distrinet bedanken voor hun steun en de aangename gesprekken doorheen het jaar.

Verder ben ik mijn vrienden en familie, Bran, Cas, Melissa, Robin, Warre, 2x Lotte, Angelo, Elise en vele anderen dankbaar voor hun vriendschap, aanmoediging en de mooie jaren samen in het hoger onderwijs.

Tot slot wil ik iedereen bedanken die mij op welke manier dan ook heeft gesteund tijdens mijn masterjaar. Jullie steun heeft het verschil gemaakt.

Ik hoop dat het onderwerp u even hard kan boeien als mij.

Milan Schollier

Gent, May 21, 2025



*Copyright informatie:  
studiewerk van een student in het kader van de  
academische opleiding en examenbeoordeling.  
Na deze beoordeling vond geen correctie plaats  
van het studiewerk.*

# Abstract

## 0.1 Abstract

In today's data-driven economy, location data holds immense value. It is used to gain meaningful crowd insights and optimize various sectors of society, ranging from tourism to retail. However, the methods used to collect these valuable location traces often occur without the user's explicit, informed consent. The high precision of location data and the frequency of reports pose significant privacy risks for individuals.

These risks include not only the inference of sensitive locations, such as home, workplace, and leisure activities, but also the construction of social graphs. In the hands of malicious actors, this information could lead to blackmail, coercion, or hate crimes.

To address these privacy concerns, this thesis proposes a privacy-friendly alternative for collecting valuable location traces. A novel Android library is developed that obfuscates both locations and timestamps by introducing noise, ensuring that individual traces can no longer be linked to a specific user. This prevents sensitive information from leaking while preserving the utility of the data. Importantly, the library operates entirely on-device, ensuring that the real, accurate traces never leave the device.

This achieve this, this thesis explores the Android ecosystem on lifecycles and background tasks to design and implement advanced location obfuscation algorithms within the library.

A set of experiments is conducted to demonstrate the effectiveness of the solution. Among other evaluations, this thesis compares the proposed library to the Approximate Location feature introduced in Android 12, offering insights into advancements in mobile privacy measures.

## 0.2 Samenvatting

In de huidige data-gedreven economie heeft locatiedata een enorme waarde. Het wordt gebruikt om waardevolle inzichten te verkrijgen over menigten en om verschillende sectoren van de samenleving te optimaliseren, variërend van toerisme tot retail.

De methoden die worden gebruikt om deze waardevolle locatiegegevens te verzamelen, gebeuren echter vaak zonder de expliciete, geïnformeerde toestemming van de gebruiker. De hoge pre-

cisie van locatiedata en de frequentie van rapportages brengen aanzienlijke privacyrisico's met zich mee voor individuen.

Deze risico's omvatten niet alleen het afleiden van gevoelige locaties, zoals woonplaats, werkplek en vrijetijdsactiviteiten, maar ook de constructie van sociale grafen. In handen van kwaadwillende actoren kan deze informatie leiden tot chantage, dwang of haatmisdrijven.

Om deze privacyproblemen aan te pakken, stelt deze thesis een privacyvriendelijk alternatief voor, voor het verzamelen van waardevolle locatiegegevens voor. Een innovatieve Android-bibliotheek is ontwikkeld die zowel locaties als tijdstempels obfusceert door ruis toe te voegen, waardoor individuele sporen niet langer aan een specifieke gebruiker kunnen worden gekoppeld. Dit voorkomt dat gevoelige informatie uitlekt, terwijl de bruikbaarheid van de data behouden blijft. Belangrijk is dat de bibliotheek volledig op het apparaat werkt, waardoor de echte, nauwkeurige gegevens het apparaat nooit verlaten.

Om dit te bereiken, onderzoekt deze thesis het Android-ecosysteem, inclusief levenscycli en achtergrondtaken, om geavanceerde algoritmen voor locatie-obfuscatie te ontwerpen en te implementeren binnen de bibliotheek. Een reeks experimenten is uitgevoerd om de effectiviteit van de oplossing aan te tonen. De thesis vergelijkt de voorgestelde bibliotheek met de Approximate Location-functie die is geïntroduceerd in Android 12, en biedt inzichten in de vooruitgang op het gebied van mobiele privacymaatregelen.

Trefwoorden: Android library, location obfuscation, location tracking

# Contents

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 0.1      | Abstract . . . . .                             | vii       |
| 0.2      | Samenvatting . . . . .                         | vii       |
| 0.3      | Probleemstelling . . . . .                     | 2         |
| 0.4      | Onderzoeksvragen . . . . .                     | 3         |
| 0.5      | Doelen en scope . . . . .                      | 4         |
| 0.6      | Overzicht van de thesis . . . . .              | 4         |
| <b>1</b> | <b>State of the art</b>                        | <b>5</b>  |
| 1.1      | Android besturingssysteem . . . . .            | 5         |
| 1.1.1    | Library . . . . .                              | 5         |
| 1.1.2    | Activity . . . . .                             | 6         |
| 1.1.3    | Achtergrond werk . . . . .                     | 8         |
| 1.2      | Locatie obfuscatie . . . . .                   | 9         |
| 1.2.1    | Android's oplossing . . . . .                  | 9         |
| 1.2.2    | Voorgaand onderzoek . . . . .                  | 10        |
| 1.3      | Preliminaire experimenten . . . . .            | 11        |
| 1.3.1    | Verschillende nauwkeurigheidsniveaus . . . . . | 11        |
| 1.3.2    | Coarse location Android . . . . .              | 12        |
| <b>2</b> | <b>Design</b>                                  | <b>15</b> |
| 2.1      | Vereisten . . . . .                            | 15        |
| 2.2      | Veronderstellingen . . . . .                   | 16        |
| 2.3      | Ontwerp . . . . .                              | 17        |
| <b>3</b> | <b>Implementatie</b>                           | <b>19</b> |
| 3.1      | Overzicht library . . . . .                    | 19        |
| 3.1.1    | Locatie tracking . . . . .                     | 19        |
| 3.1.2    | Obfuscatie . . . . .                           | 23        |
| 3.1.3    | Opslag . . . . .                               | 25        |

|          |                                                                                      |           |
|----------|--------------------------------------------------------------------------------------|-----------|
| 3.1.4    | Netwerk . . . . .                                                                    | 25        |
| 3.2      | Applicatie . . . . .                                                                 | 25        |
| 3.3      | Dataverwerking . . . . .                                                             | 27        |
| 3.3.1    | Verzameling . . . . .                                                                | 27        |
| 3.3.2    | Verwerking . . . . .                                                                 | 27        |
| 3.4      | Conclusie . . . . .                                                                  | 28        |
| <b>4</b> | <b>Experimenten en resultaten</b>                                                    | <b>29</b> |
| 4.1      | Visuele representatie . . . . .                                                      | 29        |
| 4.1.1    | Variabelen in kaart brengen . . . . .                                                | 30        |
| 4.1.2    | Uiteindelijke resultaat . . . . .                                                    | 31        |
| <b>5</b> | <b>Evaluatie en discussie</b>                                                        | <b>33</b> |
| 5.1      | Evaluatie van het systeem . . . . .                                                  | 33        |
| 5.1.1    | Beperkingen . . . . .                                                                | 33        |
| 5.2      | Verder werk . . . . .                                                                | 34        |
| 5.3      | Gebruik van de library . . . . .                                                     | 34        |
| 5.3.1    | Initialisatie . . . . .                                                              | 34        |
| 5.3.2    | LocationObfuscator zelf implementeren . . . . .                                      | 34        |
| 5.3.3    | Live data . . . . .                                                                  | 35        |
| 5.4      | Conclusie . . . . .                                                                  | 35        |
| <b>6</b> | <b>Conclusie</b>                                                                     | <b>37</b> |
| <b>A</b> | <b>Code van het obfuscatie algoritme</b>                                             | <b>41</b> |
| <b>B</b> | <b>Beschrijving van deze masterproef in de vorm van een wetenschappelijk artikel</b> | <b>43</b> |
| <b>C</b> | <b>Poster</b>                                                                        | <b>45</b> |

## List of Figures

|     |                                                         |    |
|-----|---------------------------------------------------------|----|
| 1.1 | Activity lifecycle Android[4]                           | 6  |
| 1.2 | Keuze boom achtergrond werk voor event driven tasks [5] | 7  |
| 1.3 | Android WorkManager                                     | 8  |
| 1.4 | Vergelijking coarse permissies en fine permissies 1     | 13 |
| 1.5 | Vergelijking coarse permissies en fine permissies 2     | 14 |
| 1.6 | Vergelijking coarse permissie op één apparaat           | 14 |
| 1.7 | Vergelijking coarse permissie op meerdere apparaten     | 14 |
| 2.1 | Flow diagram van continue locatieverzameling            | 17 |
| 3.1 | Klasse diagram van de library                           | 20 |
| 3.2 | Flow diagram van het obfuscatie algoritme               | 25 |
| 3.3 | Visualisatie van de test applicatie                     | 27 |
| 4.1 | Echte pad                                               | 30 |
| 4.2 | Vergelijking grootte privacycirkel                      | 31 |
| 4.3 | Vergelijking tijdsinterval locatiegegevens              | 31 |
| 4.4 | Obfuscatiealgoritme gevisualiseerd                      | 32 |





## List of Tables



# Acronyms

**AAR** Android archive. 5

**API** application program interface. 4, 8, 9, 17, 25

**APK** Android package. 5, 22

Veel moderne applicaties verzamelen locatiegegevens van hun gebruikers. Deze gegevens worden vaak gebruikt voor crowd-insights die nuttig zijn voor uiteenlopende doeleinden. Voorbeelden van onderzoeken die hierdoor mogelijk genaakt worden zijn mobiliteitsstudies en stadsplanning. Een voorbeeld hiervan is een winkelketen die wil bepalen waar een nieuwe vestiging het best kan worden gebouwd, zonder bestaande locaties te hinderen. Zo is Gravy analytics een data broker die data opkoopt en dan analyses of marktonderzoeken aanbiedt aan bedrijven. Echter zorgt hoge nauwkeurigheid en frequentie waarmee locatiegegevens worden verzameld, voor aanzienlijke privacyrisico's met zich mee. Zo zijn er altijd wel datalekken, zoals het incident bij Gravy Analytics [1]. Waarbij locatiegegevens van miljoenen gebruikers werden gelekt na een inbraak. In verkeerde handen kunnen dergelijke gegevens leiden tot sociale profilering of zelfs spionage, wat de persoonlijke veiligheid van gebruikers in gevaar brengt.

Bovendien is een datalek niet altijd nodig om privacyproblemen te veroorzaken. Onzorgvuldig omgaan met afgeleide gegevens kan eveneens risico's opleveren. Zo bleek bij datingapps dat gebruikers konden worden getrianguleerd via de functie die de afstand tot andere personen toont [2].

Door deze risico's worden zowel het publiek als overheden steeds voorzichtiger. Een voorbeeld hiervan is de overname van de populaire datingapp Grindr door een Chinees bedrijf, waarbij zorgen ontstonden over de omgang met locatiegegevens. Het Amerikaanse leger heeft het gebruik van deze app, die locatiegegevens vereist, zelfs als veiligheidsrisico bestempeld, omdat zo de locatie van soldaten en militaire bases bekend zou kunnen raken [3].

Met deze thesis wordt een bijdrage geleverd aan de ontwikkeling van privacyvriendelijke technologieën voor mobiele toepassingen. De voorgestelde oplossing zorgt ervoor dat gevoelige locatiedata zelfs nooit het apparaat verlaat waardoor vroeger in de ketting naar de data broker het probleem wordt aangepakt. Ook wilt deze thesis de instapdrempel voor ontwikkelaars drukken en de gebruiksvriendelijkheid bevorderen zodat veilig met locatiedata omgaan een standaard wordt in een wereld waarin data steeds waardevoller — en kwetsbaarder — worden.

### 0.3 Probleemstelling

Het centrale probleem is dat de werkelijke locatie van een gebruiker doorgaans rechtstreeks naar de server wordt verzonden en daar zonder enige vorm van obfuscatie wordt opgeslagen. Hierdoor blijven locatiegegevens in hun oorspronkelijke, niet-versluierde vorm beschikbaar, wat aanzienlijke privacyrisico's met zich meebrengt. Bovendien worden deze gegevens vaak zonder verdere bewerking doorverkocht aan data brokers.

In de huidige stand van zaken is privacy moeilijk te waarborgen. namelijk in de huidige stand van zaken om privacy te geranderen moet de ketting van apparaat naar data broker er als volgt uit zien.

- In de overdracht van de data naar de server moet volledig beveiligd zijn. Zo mag

het netwerkverkeer geen slachtoffer zijn van gekende zwakte zoals man-in-the-middle attack waar een hacker al het netwerkverkeer kan afluisteren.

- Eenmaal de data toekomt bij de server moet de locatiedata versluierd worden om bij eventuele datalekken de gebruikers te beschermen
- Bij het gebruik van deze data, moest het nog niet versluierd zijn, moet ook veiligheid in acht genomen worden zodat je geen triangulatie krijgt zoals bij dating apps vroeger. Want in deze stap kunnen derde partijen ook gevoelige data verzamelen.
- Dan als deze data wordt verkocht aan een data broker, en de data zou nog steeds niet versluierd zijn, moet dit zeker versluierd doorverkocht worden.
- Naast al deze veiligheidsrisico is een sociaal veiligheidsrisico waarbij de data broker geen goede intenties heeft ook van belang. Waardoor een strikte controle op de partijen die toegang krijgen tot deze gegevens noodzakelijk om misbruik te voorkomen.

De aanwezigheid van deze potentiële kwetsbaarheden bemoeilijkt het voor gebruikers om vertrouwen te hebben in de huidige verwerking van locatiegegevens.

Naast dit centraal probleem zorgen de steile leercurve van de relevante Android-technologieën en vaak voorkomende strakke deadlines voor een bijkomende uitdaging. Deze bijkomende uitdagingen kunnen hierdoor zorgen voor beveiligingslacunes in de implementatie. Een goed ontworpen bibliotheek die de complexiteit voor ontwikkelaars verlaagt en standaard voorziet in robuuste beveiligingsmaatregelen, kan hierdoor deze risico's aanzienlijk beperken.

## 0.4 Onderzoeksvragen

Het onderzoek beantwoordt 2 kernvragen: Hoe kan een Android-bibliotheek worden ontworpen die locatiegegevens lokaal en gebruiksvriendelijk obfusceert?

- Is het mogelijk voor een duidelijke abstractie zodat de developer ook eigen obfuscatie algoritmen makkelijk kan implementeren?
- Is een standaard initialisatie zo kort mogelijk? Met goed gekozen standaard instellingen.
- Hoe makkelijk is de library te implementeren in een bestaand project waar locatie verzameling aan toegevoegd zou worden?

In hoeverre beschermd de obfuscatie de locatie van de gebruiker?

- Kan een algemeen pad, dat nog nuttig is voor marktonderzoek, nog afgeleid worden uit het geobfusceerde pad?
- Is de data wel degelijk veilig in de bibliotheek?
- In hoeverre is geobfusceerde data nog nuttig voor data brokers?

## 0.5 Doelen en scope

Het doel van deze thesis is om een location data stream obfuscator te implementeren als een Android-bibliotheek. Waarbij de obfuscator locatiegegevens lokaal versluiert en dat de echte gegevens nooit het apparaat verlaten. Dit zodat individuele bewegingspatronen niet meer naar specifieke personen kunnen worden herleid. De ontwikkeling van deze bibliotheek zou de obfuscatie berekening van de server naar de client verhuizen. Waardoor gevoelige informatie (bv. een privacy bubbel) lokaal op het apparaat blijft en enkel geobfusceerde locaties worden doorgestuurd en zo data lekken minder schadelijk zijn.

In deze thesis wordt de focus uitsluitend gelegd op het lokaal verzamelen en obfusceren van locatie- en tijdgegevens op Android-apparaten. Aspecten zoals server-side beveiliging, netwerkbeveiliging en bredere privacyvraagstukken, waaronder identiteitsbeheer en gebruikersconsent, worden buiten beschouwing gelaten. De implementatie is beperkt tot het Android-platform; andere mobiele besturingssystemen, zoals iOS, vallen buiten de scope van dit onderzoek.

## 0.6 Overzicht van de thesis

**Hoofdstuk 1 application program interface (API) State of the art:** In dit hoofdstuk wordt een overzicht gegeven van de relevante aspecten van het Android besturingssysteem die van belang zijn voor deze thesis. Daarnaast wordt de huidige stand van zaken omtrent locatieverzameling en obfuscatie besproken. Ook worden enkele preliminaire experimenten toegelicht die bijdragen aan een beter begrip van de implementatie in Android, wat de basis vormt voor de keuzes die in Hoofdstuk 3 worden gemaakt.

**Hoofdstuk 2 Design:** In dit hoofdstuk worden de vereisten en veronderstellingen van de bibliotheek besproken. Daarnaast worden de belangrijkste ontwerpkeuzes en architecturale beslissingen toegelicht die aan de basis liggen van de implementatie.

**Hoofdsuk 3 Implementatie:** In dit hoofdstuk wordt de volledige werkwijze van implementatie toegelicht.

**Hoofdsuk 4 Resultaten:** In dit hoofdstuk worden visueel en numeriek de resultaten van de gemaakte library toegelicht.

**Hoofdsuk 5 Evaluatie en discussie:** Evaluatie of de bibliotheek aan de vereisten voldoet, mogelijke uitbreidingen en mogelijke verbeteringen

# 1

## State of the art

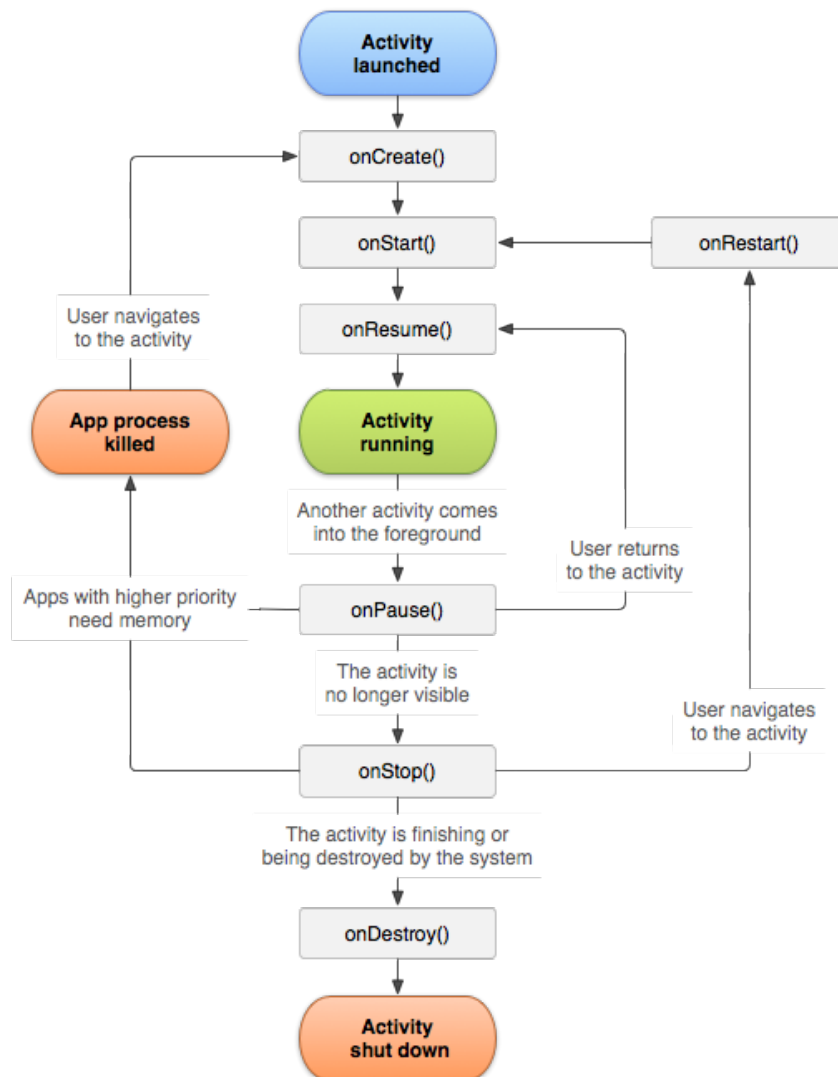
Voor de achtergrond te kaderen voor het werk van deze thesis te kunnen bespreken zijn er 2 grote stromingen: het Android besturingssysteem en locatieverzameling/-obfuscatie. Ook is er een extra sectie voorzien voor het onderzoek naar hoe de standaardimplementatie van obfuscatie werkt in Android want de documentatie bespreekt sommige dingen niet.

### **1.1 Android besturingssysteem**

#### **1.1.1 Library**

Omdat ontwikkelaars een code efficiënt zouden kunnen implementeren zonder herhaaldelijk gebruik te maken van copy-paste, worden libraries gebruikt. Android-applicaties worden voornamelijk geprogrammeerd in Java of Kotlin, waarbij de build-tool Gradle wordt ingezet. Deze build-tool beheert de installatie van de benodigde libraries, zodat deze eenvoudig beschikbaar zijn voor integratie in de eigen code. In tegenstelling tot een standaard applicatie, die wordt gecompileerd naar een Android package (APK), wordt een library voor Android gecompileerd naar een Android archive (AAR).

Deze AAR-bestanden kunnen vervolgens worden gepubliceerd naar publieke Gradle-plugins, waardoor ze eenvoudig herbruikbaar zijn in andere projecten door deze te implementeren.



**Figure 1.1:** Activity lifecycle Android[4]

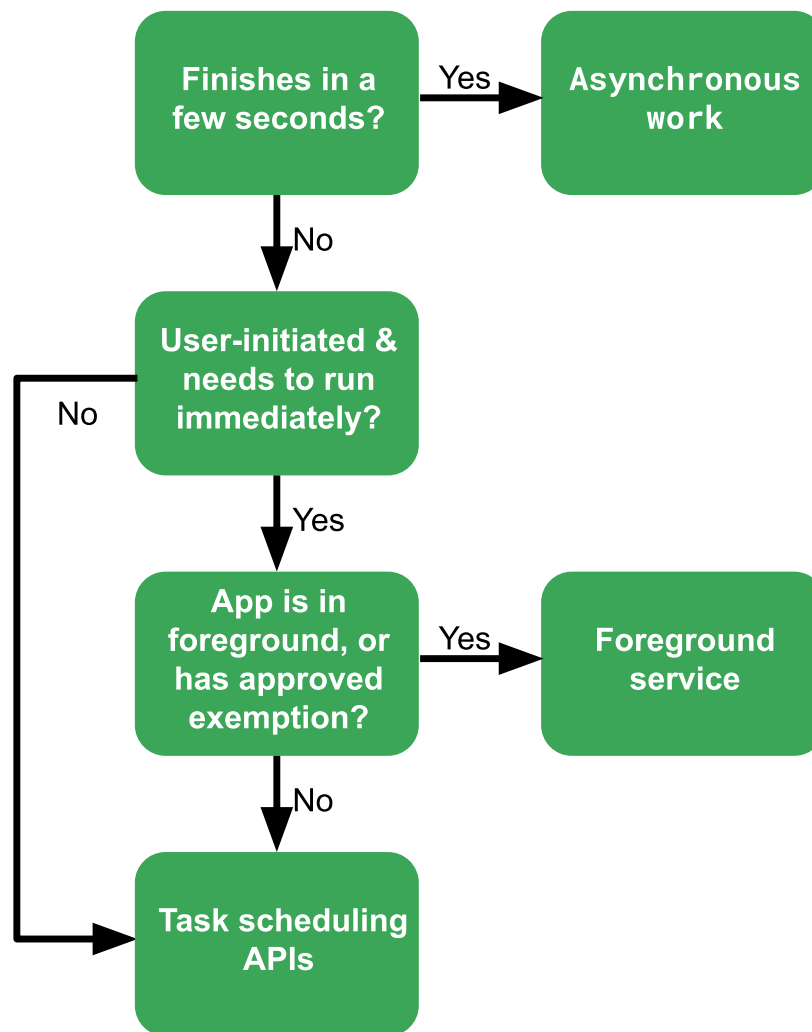
### 1.1.2 Activity

Een activity in Android is een enkele, gefocuste taak die een gebruiker kan uitvoeren. Het vormt een van de fundamentele bouwstenen van een Android-applicatie. Elke activity heeft een eigen lifecycle, die wordt beheerd door het Android-systeem. Deze lifecycle bestaat uit verschillende states, zoals onCreate, onStart, onResume, onPause, onStop en onDestroy.

Elke state heeft ook een methode die de ontwikkelaar kan overschrijven om zo de overgang beter te kunnen afhandelen.

Bij het ontwikkelen van een applicatie is het belangrijk om rekening te houden met deze lifecycle-methoden, omdat ze bepalen hoe de applicatie reageert op gebruikersinteracties en systeemgebeurtenissen. Bijvoorbeeld, in de onCreate-methode wordt de initiële setup van de activity uitgevoerd, zoals het instellen van de gebruikersinterface. De onPause- en onStop-





**Figure 1.2:** Keuze boom achtergrond werk voor event driven tasks [5]

methoden worden aangeroepen wanneer de activity naar de achtergrond gaat, en resources kunnen hier worden vrijgegeven om geheugen te besparen.

Zo heeft elke state bepaalde eigenschappen. Zo worden langdurige taken die in de states draaien, waarbij de activity naar de achtergrond gaat, abrupt gestopt als het systeem resources nodig heeft en is dus niet betrouwbaar. Om dit probleem te omzeilen biedt Android alternatieve oplossingen zoals services(waarover volgende hoofdstuk meer). Daarnaast is het niet toegestaan om blocking calls, zoals een HTTP-request of het opvragen van locatiegegevens via de Location Provider, uit te voeren in de main thread. Dit is noodzakelijk om te voorkomen dat de gebruikersinterface vastloopt en de gebruikservaring negatief wordt beïnvloed. Zo crashed de applicatie als de main thread wordt geblocked voor 5 seconden.

### 1.1.3 Achtergrond werk

Om de hierboven vermelde beperkingen van een activity, die slechts kort op de achtergrond actief kan blijven, te omzeilen. Biedt Android verschillende tools die door de jaren heen zijn geëvolueerd. Deze tools maken het mogelijk om taken los van de activity uit te voeren, zelfs wanneer de applicatie niet actief is. Maar elke API is verschillend om zo elk probleem aan te pakken. Hier focussen we op de keuze boom gerelateerd voor het starten van een achtergrond taak gebaseerd op een event. Dit omdat de optie om een eenmalige locatie op te vragen in de applicatie kan afgehandeld worden door een simpele thread.

Zoals te zien in figuur 1.2 is asynchrone werk niet van toepassing voor deze thesis. Dan heb je nog Task scheduling en Foreground services. Hierin zijn de belangrijkste tools zijn de AlarmManager en de WorkManager.

**AlarmManager** [6] Alarm manager is een implementatie van een task scheduling API De AlarmManager is een mechanisme dat het apparaat uit de zogenaamde 'doze mode' of energiebesparende modus kan halen. Dit maakt het mogelijk om op specifieke tijdstippen een taak uit te voeren. Echter, de AlarmManager heeft enkele beperkingen. Zo kan het slechts één keer per uur een taak uitvoeren en is het niet geschikt voor langdurige taken. Dit maakt het minder geschikt voor toepassingen die continu of frequent achtergrondtaken moeten uitvoeren. AlarmManager wordt dikwijls gebruikt om persistent achtergrondwerk op te starten als de applicatie niet opgestart is.

**WorkManager** [7] De WorkManager is een krachtig en flexibel API voor het uitvoeren van

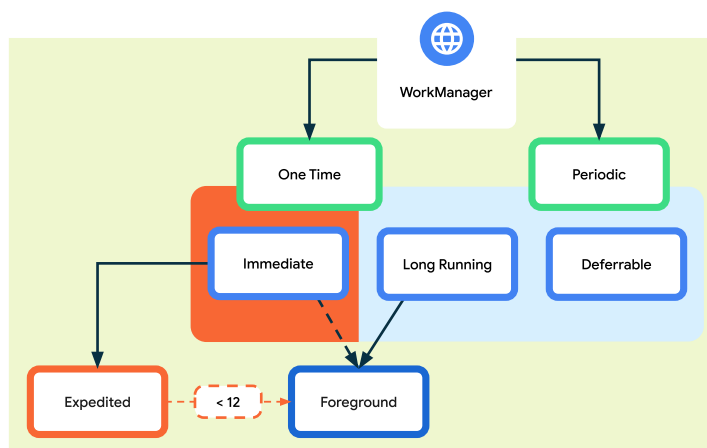


Figure 1.3: Android WorkManager

taken die niet gekoppeld zijn aan een activity. Het is zowel een oplossing voor foreground services als voor task scheduling. Zo biedt ondersteuning voor zowel eenmalige langdurige (one-time) als periodieke (periodic) taken. Voor achtergrondtaken die regelmatig moeten

worden uitgevoerd, zoals het verzamelen van locatiegegevens, is de periodieke versie geschikt. Hierbij kan een taak bijvoorbeeld elke 15 minuten worden uitgevoerd, met een maximale duur van 10 minuten per taak. Voor taken die slechts één keer hoeven te worden uitgevoerd, zoals een langdurige upload, kan de eenmalige versie worden gebruikt. Het is echter een belangrijk punt om op te merken dat langdurige taken in de foreground een permanente notificatie in het besturingssysteem vereisen, wat de gebruiker kan storen.

**Coroutines** [8] Coroutines vormen geen API voor het aanbieden van persistente services, maar zijn wel de aangewezen oplossing voor het uitvoeren van zware of blocking taken buiten de hoofdthread van een activity. Coroutines kunnen worden beschouwd als lichtgewicht alternatieven voor threads en bieden een efficiënte manier om asynchrone taken uit te voeren zonder de gebruikersinterface te blokkeren. Ze zijn echter gekoppeld aan de lifecycle van de aanroepende component (zoals een activity of fragment). Dit betekent dat wanneer de activity wordt gestopt, ook de bijbehorende coroutines worden beëindigd. Hierdoor zijn coroutines niet geschikt voor persistent achtergrondwerk dat moet blijven draaien, ongeacht de status van de activity. Dit zou dus bedoeld zijn om een eenmalige locatie op te vragen.

## 1.2 Locatie obfuscatie

In de studie van obfuscatie heb je 2 paden: hoe Android het aanpakt en voorgaand onderzoek losstaand van Android.

### 1.2.1 Android's oplossing

Voordat er dieper wordt ingegaan op locatie-obfuscatie in Android, is het belangrijk om de werking van de locatievoorzieningen in Android te bespreken. Android biedt twee primaire opties om locatiegegevens op te vragen: de standaard open-source implementatie en de Google Play Services-implementatie.

De standaard open-source implementatie leest enkel de data uit van de sensoren en voert geen verdere berekeningen of obfuscatie-algoritmen uit. Deze aanpak biedt ontwikkelaars meer vrijheid, maar brengt ook enkele nadelen met zich mee. Zo worden applicaties die deze versie gebruiken vaker gemarkeerd als potentieel kwaadaardig, omdat ze minder geïntegreerd zijn in het Google Android-ecosysteem.

De Google Play Services-implementatie, daarentegen, maakt gebruik van de Fused Location Provider. De Fused Location Provider combineert gegevens van verschillende sensoren, zoals GPS, wifi, mobiele netwerken en zelfs bewegingssensoren, om een zo nauwkeurig mogelijke locatie te berekenen. Dit proces wordt 'sensor fusion' genoemd. De Fused Location Provider biedt ontwikkelaars een API om locatiegegevens op te vragen, waarbij automatisch de meest geschikte bron wordt gekozen op basis van de context, zoals batterijverbruik, nauwkeurigheid en beschikbaarheid van sensoren. Bovendien ondersteunt de Fused Location

Provider zowel real-time locatie-updates als eenmalige locatieverzoeken, wat het een flexibele oplossing maakt voor uiteenlopende toepassingen.

In de Fused Location Provider zijn er verschillende methoden om locatiegegevens te verkrijgen. De meest nauwkeurige methode is via GPS-satellieten, maar alternatieve methoden zoals het gebruik van lokale telefoonmasten of wifi-netwerken bieden ook mogelijkheden. Deze alternatieve methoden introduceren een inherente vorm van obfuscatie, aangezien de nauwkeurigheid doorgaans lager is dan die van GPS. Echter, in dichtbevolkte stedelijke gebieden kan dit juist leiden tot een hogere nauwkeurigheid dan GPS, omdat gebouwen GPS-signalen kunnen blokkeren.

Sinds Android 12 hebben gebruikers de mogelijkheid gekregen om alleen toegang tot een benaderde locatie (approximate location) toe te staan, wat een extra dimensie toevoegt aan de discussie over privacy en nauwkeurigheid. De coarse location-functionaliteit in Android maakt geen gebruik van GPS-satellieten en voegt bovendien een extra laag van obfuscatie toe aan de locatiegegevens. Hoe deze obfuscatie precies wordt toegepast, is echter niet publiekelijk gedocumenteerd.

### 1.2.2 Voorgaand onderzoek

**Aanval van Kevin De Boeck** In eerdere onderzoeken, uitgevoerd door Kevin De Boeck, werd aangetoond dat locatie-obfuscatie een dringende noodzaak is. Uit data-analyse op een dataset van locatiegegevens in Frankrijk, verzameld over een periode van zeven dagen, bleek dat gevoelige locaties eenvoudig konden worden geïdentificeerd. Ook op vlak van het individu kon makkelijk een thuis of andere persoonlijke locaties gevonden worden. Verder konden er sociale grafen gemaakt worden en zelfs een volledige vakantie reis in kaart gebracht worden.

**Oplossing Toon Dehaene** Voortbouwend op deze bevindingen heeft Toon Dehaene onderzoek gedaan naar methoden om GPS-locaties te obfusceren. Hiervoor werd een library ontwikkeld in Rust met behulp van PyO3, zodat deze als Python-module kon worden gebruikt voor het verwerken van grote datasets. Het doel was om locatie-traces op serverzijde te obfusceren. Naast de implementatie van parallelisatie was het kernalgoritme voor obfuscatie als volgt:

Wanneer er onvoldoende tijd is verstreken sinds de vorige locatie, wordt simpelweg de vorige locatie opnieuw doorgestuurd. Vervolgens wordt het concept van privacycirkels toegepast: als de nieuwe locatie binnen een bestaande privacycirkel valt, wordt het middelpunt van deze cirkel samen met de huidige tijdstempel doorgestuurd. Valt de locatie buiten alle bestaande cirkels, dan wordt er een nieuwe cirkel aangemaakt met een willekeurige hoek en een straal die willekeurig gekozen wordt tussen 0 en de huidige maximale straal. De gebruiker valt dan altijd binnen deze nieuw gegenereerde cirkel, waarvan het middelpunt wordt doorgestuurd.

Dit algoritme introduceert dus ruis op de coördinaten en, belangrijker nog, voorkomt dat derden door het nemen van gemiddelden van geobfusceerde locaties alsnog gevoelige locaties

kunnen achterhalen. Het algoritme vormde tevens de basis voor de standaardimplementatie van obfuscatie in de Android-library, wat verderop wordt toegelicht.

**Testen van de oplossing** In verder onderzoek van Toon Dehaene werd nagegaan of deze geobfusceerde locaties konden worden teruggevonden via gridsnapping. Dit werd onderzocht om de effectiviteit van locatieobfuscatie in afgelegen gebieden te evalueren. Wanneer er zich slechts één straat binnen de straal van een privacycirkel bevindt, is het immers visueel eenvoudig om de oorspronkelijke route te reconstrueren. Dit roept de vraag op of de straal van de privacycirkel dynamisch zou moeten worden aangepast op basis van de bebouwingsdichtheid. Een bijkomende onderzoeksvraag was of het mogelijk is om een obfuscatiemethode te ontwikkelen waarbij de geobfusceerde locatie bewust op een willekeurige, maar plausibele locatie binnen een grid wordt geplaatst, zodat de geobfusceerde locaties in afgelegen gebieden realistischer lijken. Uit de resultaten bleek echter dat deze aanpak een averechts effect had. Verder onderzoek naar deze methode valt buiten de scope van deze thesis.

## 1.3 Preliminare experimenten

Aangezien de Android Developer-website niet alle vragen beantwoorde en er nog veel onduidelijkheden zijn, is een deel van de state-of-the-art gewijd aan onderzoek naar hoe deze variabelen het uiteindelijke resultaat beïnvloeden.

### 1.3.1 Verschillende nauwkeurigheidsniveaus

De Fused Location Provider biedt vier prioriteitsniveaus voor het bepalen van de locatie, zoals beschreven in de Android-documentatie [9]:

- **Balanced power accuracy:** Gebruik deze instelling om locatieprecisie op te vragen binnen een stadsblok, wat een nauwkeurigheid van ongeveer 100 meter is. Dit wordt beschouwd als een grove nauwkeurigheid en verbruikt waarschijnlijk minder stroom. Met deze instelling maken de locatieservices waarschijnlijk gebruik van Wi-Fi en positionering van zendmasten. Houd er echter rekening mee dat de keuze van de locatieprovider afhankelijk is van veel andere factoren, zoals welke bronnen beschikbaar zijn.
- **High accuracy:** Gebruik deze instelling om de meest nauwkeurige locatie op te vragen. Met deze instelling is de kans groter dat de locatieservices GPS gebruiken om de locatie te bepalen.
- **Low power:** Gebruik deze instelling om precisie op stadsniveau aan te vragen, wat een nauwkeurigheid van ongeveer 10 kilometer is. Dit wordt beschouwd als een grove nauwkeurigheid en verbruikt waarschijnlijk minder stroom.

- **Passive:** Gebruik deze instelling als u een verwaarloosbare invloed op het stroomverbruik nodig heeft, maar locatie-updates wilt ontvangen wanneer deze beschikbaar zijn. Met deze instelling activeert uw app geen locatie-updates, maar ontvangt u locaties die door andere apps worden geactiveerd.

Naast de beschreven verschillen in nauwkeurigheid en energieverbruik, zijn er ook andere factoren die deze instellingen beïnvloeden. Uit experimenten bleek bijvoorbeeld dat de instelling *balanced power accuracy* wordt beïnvloed door de doze-modus of energiebesparingsmodus van Android. Tijdens tests werd vastgesteld dat een actief gebruikte telefoon een consistente locatie-trace genereerde, terwijl drie andere telefoons aanzienlijke hiaten vertoonden. Deze hiaten werden veroorzaakt doordat de Fused Location Provider de callback niet activeerde vanwege de energiebesparende staten.

Tests met de prioriteiten *low power* en *passive* werden niet uitgevoerd. De *low power*-instelling was niet relevant voor onderzoek naar gebruikerslocatie-traces. Dit doordat de beloofde nauwkeurigheid 10km is. De *passive*-instelling dan niet omdat er te veel externe variabelen waren, waardoor geen waardevolle resultaten konden worden verkregen. In voornameerste omdat de locatieupdates afhankelijk zijn van wat andere applicaties doen.

### 1.3.2 Coarse location Android

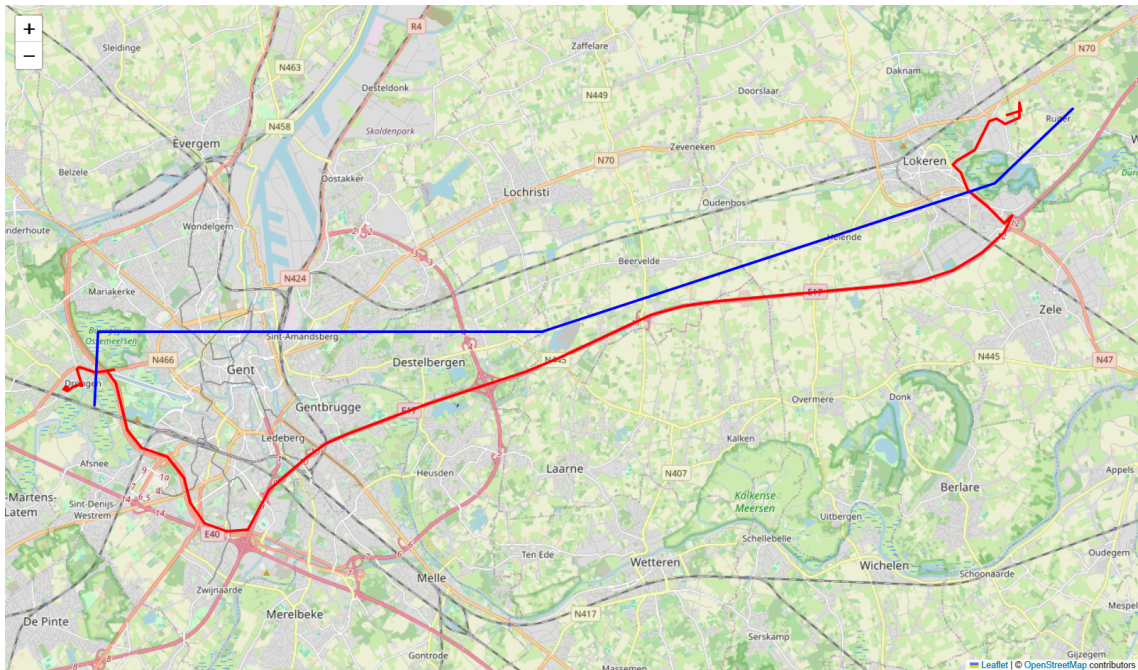
Om de coarse location-functionaliteit te onderzoeken, werd de applicatie tweemaal geïnstalleerd op dezelfde telefoon: één keer met coarse location-permissies en één keer met fine location-permissies. Beide applicaties waren geconfigureerd met dezelfde prioriteitsinstellingen om een eerlijke vergelijking mogelijk te maken. Een van de vragen was of de instelling *balanced power accuracy*, die geen gebruik maakt van satellieten, dezelfde resultaten zou opleveren als coarse location. Dit zou inzicht geven in de vraag of de obfuscatie uitsluitend voortkomt uit de inherente onnauwkeurigheid van de gebruikte technologieën, of dat Android actief een extra laag van obfuscatie toepast op de locatiegegevens. Als resultaat vinden we de volgende visualisatie ?? en 1.5. Hierbij is rood de trace met fine permissies en blauw coarse permissies.

- In beide voorbeelden zien we dat de coarse trace duidelijk minder punten bevat en verder van de werkelijke trace ligt dan de fine trace. Dit komt doordat, wanneer de nieuwe locatie te dicht bij de vorige ligt, de callback eenvoudigweg niet wordt opgeroepen.
- In het tweede voorbeeld zien we dat bij langdurig op dezelfde locatie blijven, de fine permissie relatief veel punten oplevert, waarbij de data varieert en zelfs een uitschieter vertoont. Dit is waarschijnlijk het gevolg van meer interferentie op de signalen in een betonnen gebouw. Opvallend is dat wanneer deze uitschieter optreedt, de coarse locatie eveneens een uitschieter in dezelfde richting vertoont. In dat geval wordt de callback dus wel opgeroepen.
- In het eerste voorbeeld zien we dat het coarse pad slechts vijf punten bevat. Ook ligt

het pad meer dan de helft van de breedte van Gent verwijderd van de werkelijke trace. De obfuscatie-oplossing van Android kan niet beïnvloed worden door de gebruiker.

- In het tweede voorbeeld zien we linksboven op de Gentse Steenweg bij de Colruyt dat, wanneer er een volgend punt wordt berekend door de coarse-methode, dit punt ver genoeg van de vorige punten ligt. Dit kan mogelijk een extra privacymaatregel zijn.

Uit deze bemerkingen kunnen we concluderen dat bovenop de inherente onnauwkeurigheid ook nog extra obfuscatie gebeurt. Een interessante vraag die voortkwam uit deze experimenten,

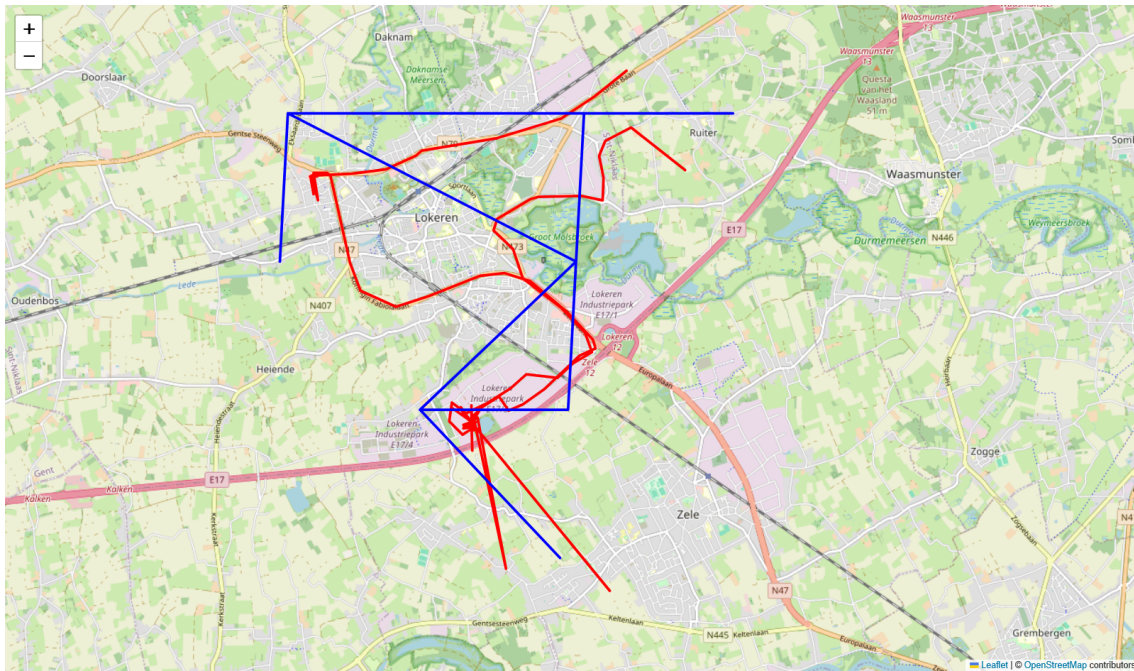


**Figure 1.4:** Vergelijking coarse permissies en fine permissies 1

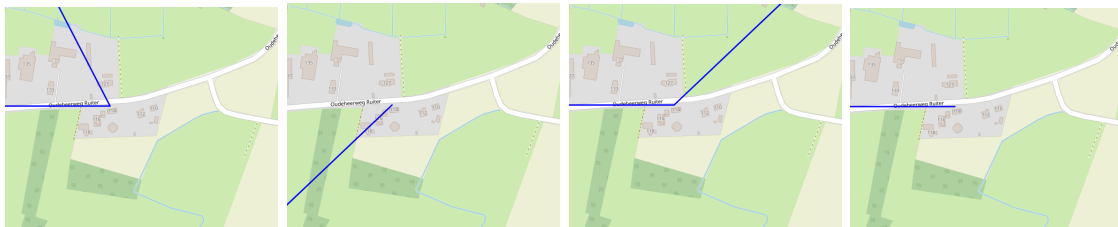
was hoe de obfuscatie van Android precies werkt. Om dit te onderzoeken, werden de coarse location-gegevens van eerdere tests naast elkaar gelegd. Zoals te zien is in Figuur 1.6, leveren de traces van op verschillende dagen maar wel hetzelfde apparaat steeds dezelfde coördinaten op. Dit wijst erop dat Android een soort privacycirkels implementeert, vergelijkbaar met eerder besproken algoritmen, om aanvallen te voorkomen waarbij gemiddelden over lange periodes worden gebruikt om plaatsen van interesse te identificeren.

Een vervolgonderzoek richtte zich op de consistentie van dit algoritme. De vraag was of de geobfusceerde locaties consistent worden berekend op basis van nabijgelegen telefoonmasten, zodat dezelfde locaties op verschillende apparaten identiek zouden zijn. Uit de resultaten, weergegeven in Figuur 1.7, blijkt echter dat dit niet het geval is. Dit suggereert dat Android's obfuscatie-algoritme niet uitsluitend gebaseerd is op vaste berekeningen van telefoonmastlocaties, maar mogelijk ook andere factoren in overweging neemt.

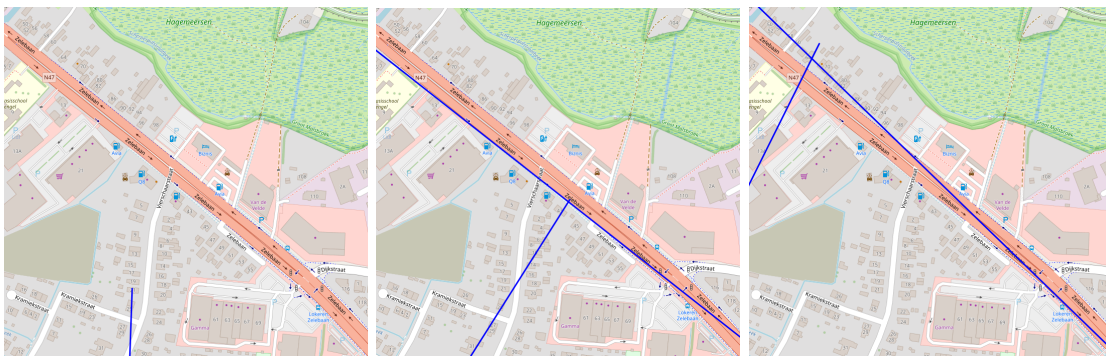




**Figure 1.5:** Vergelijking coarse permissies en fine permissies 2



**Figure 1.6:** Vergelijking coarse permissie op één apparaat



**Figure 1.7:** Vergelijking coarse permissie op meerdere apparaten



# 2

## Design

Vooraleer de implementatie van de library kan starten, is een grondig ontwerp noodzakelijk. Dit ontwerp wordt in belangrijke mate bepaald door de beperkingen en mogelijkheden van het Android-platform. Daarnaast zijn de functionele vereisten en onderliggende veronderstellingen essentieel voor het uiteindelijke ontwerp.

### 2.1 Vereisten

De doelstellingen van dit onderzoek zijn als volgt geformuleerd:

**Library** Het ontwikkelen van een library met algemene bruikbaarheid: Het streven is om een library te creëren die in diverse applicaties kan worden toegepast. Deze library moet het proces van het aanvragen van permissies, initialisatie, gebruik en uitbreiding zo efficiënt mogelijk ondersteunen. Dit impliceert dat de library aan onderstaande vereisten moet voldoen om in uiteenlopende soorten applicaties te kunnen werken.

- **Flexibel:** De library mag niet beperkend zijn in vergelijking met de standaardinstellingen van de Android-implementatie en moet voldoende configuratiemogelijkheden bieden.
- **Modulair:** Gebruikers moeten in staat zijn om hun eigen obfuscatiealgoritmen te ontwikkelen en te integreren. De code dat ze niet horen te veranderen hoort louter een facade te zijn.
- **Eenvoudig te integreren:** De library moet functioneren met minimale setup en gebruiksvriendelijke integratiemogelijkheden bieden. Met minimale setup wordt bedoeld

dat de library volledig moet functioneren met zo weinig mogelijk lines of code.

- **Lage leercurve:** Een van de aannames is dat de ontwikkelaar mogelijk weinig tot geen kennis heeft van locatie services in Android. Daarom moeten de standaardinstellingen een solide basis bieden voor het veilig omgaan met locatiegegevens.

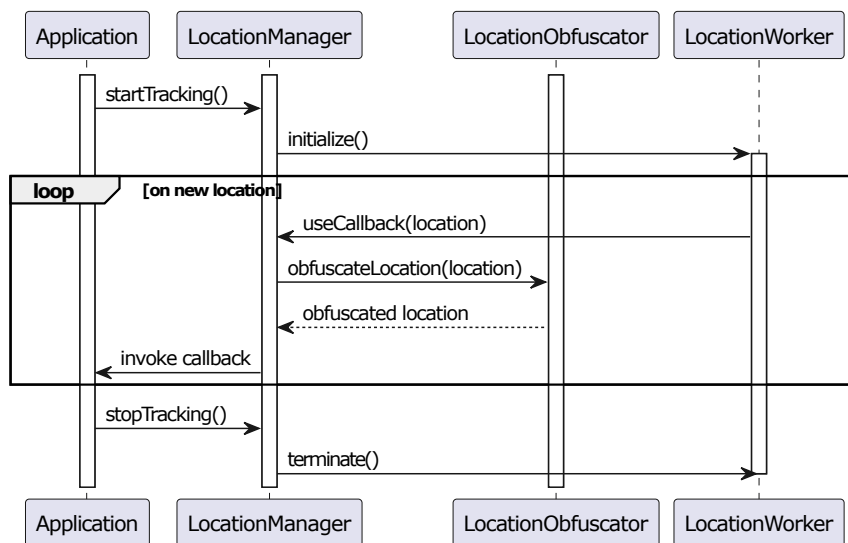
**Privacy** Op het vlak van privacy geldt als vereiste dat alle gevoelige data het toestel niet verlaat. Verder dat de data dus niet toegankelijk is voor andere applicaties.

Ook moet de geobfusceerde locatie voldoende accuraat blijven voor het beoogde gebruik, zonder dat het mogelijk is een individu te identificeren of te traceren. Voor deze obfuscatietechniek wordt voortgebouwd op het werk van Toon Dehaene, zoals eerder besproken. Dit algoritme zal moeten aangepast worden om geschikt te zijn voor verwerking van datastromen in plaats van parallelle verwerking, en zal geïmplementeerd moeten worden in Kotlin.

## 2.2 Veronderstellingen

Om de werking van de library te kunnen garanderen, worden de volgende veronderstellingen gemaakt:

- **Goede intenties van de ontwikkelaar:** Er wordt aangenomen dat de ontwikkelaar te goeder trouw handelt. De library kan immers aangepast of omzeild worden, waardoor het alsnog mogelijk is om de echte locatie door te sturen.
- **Integriteit van het Android-systeem:** Er wordt verondersteld dat het Android-systeem waarop de applicatie draait niet gecompromitteerd is. Dit betekent dat er geen derde partij of de eigenaar roottoegang heeft, aangezien dit zou toelaten om privacygevoelige gegevens rechtstreeks uit te lezen.
- **Correct gebruik van onderzoeks- en debugmodus:** De applicatie zal een onderzoeks- en debugmodus bevatten. Het is essentieel dat deze modus niet in productieomgevingen wordt gebruikt, omdat dit zou kunnen leiden tot het versturen van de echte locatie samen met de geobfusceerde locatie.
- **Normaal gebruik door de gebruiker:** De permanente opslag van gegevens wordt gekoppeld aan de installatie van de applicatie. Bij frequente herinstallatie van de applicatie worden voorgaande privacycirkels verwijderd. Dit zou het mogelijk maken om een gemiddelde te nemen van deze nieuwe privacycirkels, waardoor alsnog gevoelige locaties achterhaald kunnen worden.



**Figure 2.1:** Flow diagram van continue locatieverzameling

## 2.3 Ontwerp

Het ontwerp van de library is gebaseerd op het principe van een facade, waarbij de library fungeert als een vereenvoudigde interface voor de standaard Android-functionaliteiten. Dit betekent dat alle variabelen en functionaliteiten aanwezig moeten zijn alsof de ontwikkelaar rechtstreeks met de standaard Android-implementatie werkt, maar met de toegevoegde voordelen van de library. Met bovenstaande ontwerpprincipes in gedachten illustreert Figuur 2.1 hoe één enkele functie het verzamelen van locatiegegevens kan starten. Het is belangrijk op te merken dat deze implementatie de functie `RequestPermission` niet omvat, aangezien deze losstaand moet opgeroepen geweest zijn. Want in de flow diagram staan de Preliminaire checks niet, als de applicatie geen toegang heeft tot locaties zal deze niet proberen locaties op te vragen. Figuur 2.1 toont vier actoren. De applicatie zelf valt buiten de controle van de library. De `LocationManager` fungeert als het centrale aanspreekpunt dat de volledige werking coördineert. De `LocationObfuscator` vormt het modulaire onderdeel dat door de ontwikkelaar naar wens kan worden aangepast of vervangen. Tot slot is er de `LocationWorker`, die als aparte klasse wordt geïmplementeerd omdat deze de `Worker` van de `WorkManager`-API van Android moet uitbreiden. Deze scheiding zorgt voor een duidelijke structuur en maakt het systeem flexibel en uitbreidbaar.

Verder is de `LocationManager` een Singleton [10] gemaakt. Singletons zorgen ervoor dat er slechts één instantie van een klasse bestaat binnen de applicatie. Zodat wanneer de developer twee-maal de `LocationManager` probeert te initialiseren de applicatie niet eventueel crashed doordat beide instanties locaties opvragen en het limiet overschrijden.

Daarnaast moeten alle standaardinstellingen en functionaliteiten van de library een solide basis bieden, zodat ontwikkelaars zonder uitgebreide kennis van Android-locatieservices de library direct kunnen gebruiken. Dit omvat het bieden van een goede standaardimplementatie die

flexibel genoeg is om aanpasbaar te zijn voor meer geavanceerde toepassingen. Ook moeten de setters rekening houden met de grenzen van het Android systeem. Bijvoorbeeld als de applicatie enkel coarse permissies heeft mag de app niet frequenter dan één keer om de 2 minuten een locatie vragen of de volledige FusedLocationProvider zal niet meer werken.

# 3

## Implementatie

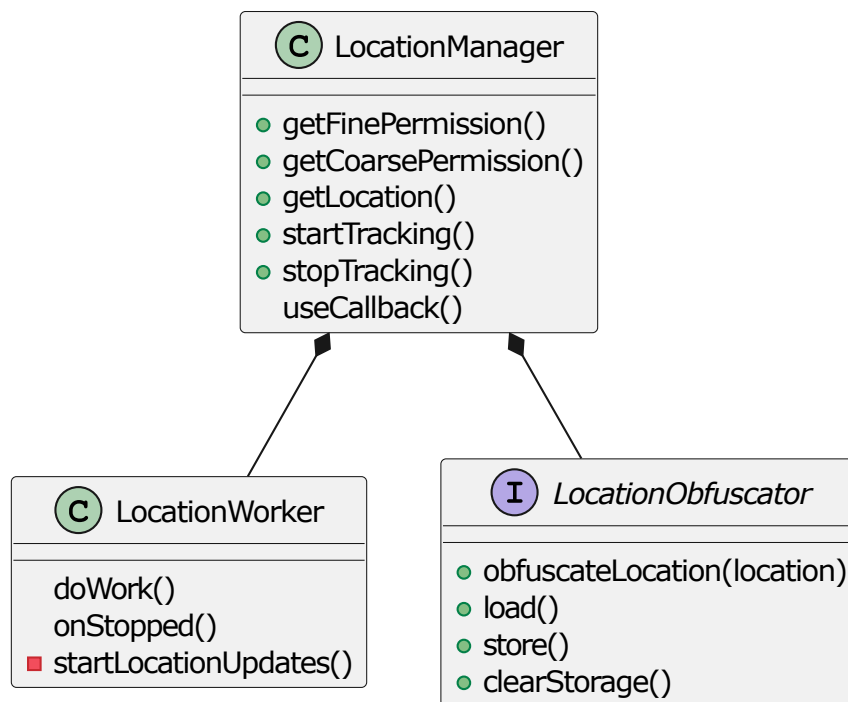
De eerste stap in deze thesis was het onderzoeken van hoe Android omgaat met locatiegegevens en het implementeren van deze functionaliteit in een testapplicatie. Zodra deze applicatie functioneel was, werden de eerste preliminaire testen uitgevoerd en werd de nuttige code afgesplitst om de basis te vormen van de Android-library.

### 3.1 Overzicht library

De structuur van de library is weergegeven in Figuur 3.1. Het merendeel van de functionaliteit is ondergebracht in de klasse `LocationManager`, die fungeert als het centrale beheerpunt voor locatieverzameling. De klasse `LocationWorker` vormt de implementatie die vereist is om de `WorkManager` te benutten voor achtergrondtaken. Daarnaast biedt de interface `LocationObfuscator` een abstractielaag waarmee ontwikkelaars eigen obfuscatiealgoritmen kunnen implementeren.

#### 3.1.1 Locatie tracking

Het centrale onderdeel van de library, namelijk de locatieverzameling, wordt beheerd door de `LocationManager`, die fungeert als het centrale aanspreekpunt. Deze klasse beheert alle benodigde variabelen voor locatieverzameling en biedt setters met ingebouwde validaties om de consistentie te waarborgen. Variabelen die specifiek zijn voor obfuscatie worden echter beheerd door de obfuscator zelf, aangezien deze buiten de verantwoordelijkheid van de `LocationManager` vallen.



**Figure 3.1:** Klasse diagram van de library

### 3.1.1.1 Background en foreground services

Voordat locaties verzameld kunnen worden, moeten enkele beperkingen in acht worden genomen. Allereerst mogen locaties niet worden opgevraagd in dezelfde thread als de activity waarmee de gebruiker interacteert. Zelfs als dit in een aparte thread vanuit de hoofdactivity wordt opgestart. Is dit geen oplossing voor permanente locatieverzameling want Android zal deze beëindigen vanwege de strikte beperkingen op services wanneer de activity wordt gestopt of vernietigd.

Om taken uit te voeren wanneer de hoofdactivity niet actief is, kan gebruik worden gemaakt van foreground services of background tasks. Voor deze library is gekozen voor de WorkManager, vanwege de flexibiliteit en betrouwbaarheid die deze biedt. De WorkManager kan zowel functioneren als een foreground service als een periodieke background task, wat het beheer van locatieverzoeken aanzienlijk vereenvoudigt.

**Foreground services** Een foreground service wordt gebruikt om langdurige taken uit te voeren die een hogere prioriteit vereisen. Om een service als foreground service te configureren, moet in de hoofdactiviteit een notificatiekanaal worden aangemaakt. Dit notificatiekanaal zorgt ervoor dat de gebruiker visueel geïnformeerd wordt over de achtergrondactiviteit. Nadat het notificatiekanaal is aangemaakt, kan de Worker, een klasse die de vereiste interface van de WorkManager implementeert, een notificatie genereren en weergeven. Dit gebeurt als volgt in de `LocationManager`, zoals weergegeven in Code ??, die een referentie heeft naar de

hoofdactiviteit. De `LocationManager` maakt een notificatiekanaal aan met `IMPORTANCE_LOW` om de gebruiker zo min mogelijk te storen.

```
private fun setupNotificationChannel() {
    val channelId = "locationTrackingChannel"
    val channelName = "Location Tracking"
    val channelDescription = "Location tracking is active"
    val notificationManager =
        activity.getSystemService(NOTIFICATION_SERVICE) as NotificationManager

    // Create notification channel if not already created
    val channel =
        NotificationChannel(channelId, channelName, NotificationManager.IMPORTANCE_LOW) {
            description = channelDescription
        }
    notificationManager.createNotificationChannel(channel)
}
```

Dan kan de `LocationWorker` ingrijpen op dit notificatiekanaal door dezelfde Id aan te spreken. De nodige info is een notificatie voor de `foregroundInfo`, zodat Android weet waarvoor de notificatie bedoeld is. De implementatie Hiervoor is te zien in code ??

```
val notification =
    NotificationCompat.Builder(applicationContext, "locationTrackingChannel")
        .setContentTitle("Location Tracking")
        .setTicker("Location Tracking")
        .setContentText("Tracking your location in the background")
        .setSmallIcon(R.drawable.ic_launcher_background)
        .setOngoing(true)
        .build()

val foregroundInfo =
    ForegroundInfo(1001, notification, ServiceInfo.FOREGROUND_SERVICE_TYPE_LOCATION)
setForegroundAsync(foregroundInfo)
```

Zodra de notificatie is geactiveerd, kan de `Worker` ononderbroken blijven werken totdat deze zichzelf beëindigt of totdat de functie `cancelUniqueWork` van de `WorkManager` wordt aangeroepen vanuit de hoofdactivity. Deze aanpak zorgt ervoor dat de taken betrouwbaar en efficiënt worden uitgevoerd, zelfs wanneer de applicatie zich niet op de voorgrond bevindt.

**Background tasks** Wanneer een visuele indicatie van achtergrondactiviteit niet vereist of mogelijk is, kan gebruik worden gemaakt van periodieke taken. Historisch gezien werd hiervoor

de `AlarmManager` gebruikt, die functioneerde als een soort cron job in Linux. Echter, vanwege strengere beperkingen op maximale duur en frequentie, is deze aanpak minder geschikt geworden.

De `WorkManager` biedt een modern en aanbevolen alternatief voor het uitvoeren van periodieke taken. Er zijn echter enkele beperkingen: taken kunnen maximaal één keer per 15 minuten worden uitgevoerd en mogen niet langer dan 10 minuten actief zijn. Ondanks deze restricties is de `WorkManager` robuust, energiezuinig en goed geïntegreerd met het Android-platform, waardoor achtergrondprocessen betrouwbaar beheerd kunnen worden. Qua implementatie wordt dezelfde codebasis gebruikt als voor de foreground service, maar zonder het opzetten van het notificatiekanaal zoals hierboven beschreven.

**Integratie in de library** Doordat de `WorkManager` beide vereisten kan afhandelen, is er slechts één klasse nodig die zowel foreground services als periodieke taken implementeert. Deze klasse, die de `Worker`-interface implementeert, bevat tevens de `LocationCallback` die vereist is voor de `FusedLocationProvider`. Deze callback speelt alle resultaten direct door naar de `LocationManager`, die deze resultaten vervolgens verwerkt volgens de ingestelde configuratie. Deze aanpak minimaliseert de complexiteit en verhoogt de herbruikbaarheid van de code.

Een bijkomende reden om de `LocationManager` als een Singleton te implementeren, is de beperking van de `LocationWorker` om complexe structuren of referenties door te geven. Zo is de initialisatie van een `Worker` beperkt tot primitieve datatypes. Hierdoor kan er geen referentie doorgegeven worden naar een stop signaal. Hiervoor komt een verder voordeel van de `LocationManager` een Singleton te maken naar boven. Want door gebruik te maken van een Singleton-patroon kan de `LocationWorker` de `LocationManager` initialiseren en een referentie behouden naar diens variabelen. Zo bevat de `LocationManager` bijvoorbeeld een boolean genaamd `running`, waarmee kan worden aangegeven of de locatieverzameling actief is. Indien de locatieverzameling wordt stopgezet, kan de `LocationWorker` deze status uitlezen en zichzelf correct beëindigen.

### 3.1.1.2 Permissies

voordeel van Android is sandboxing, waardoor alles waar de applicatie probeert toegang tot te krijgen gemonitord kan worden. Hierdoor is het noodzakelijk dat de applicatie expliciet toestemming vraagt voor bepaalde functionaliteiten; indien deze toestemming niet wordt verkregen, zal de applicatie bij een toegangsverzoek crashen. Alle benodigde permissies horen ook in het manifestbestand vermeld te worden. Dit manifest bestand is vergelijkbaar aan een `package.json` voor een nodejs project. Hierin staan alle libraries dat de applicatie nodig heeft, maar dus ook de permissies die de app eventueel kan opvragen en de services dat de app eventueel gebruik van kan maken. Dit is nodig om de applicatie te laten compileren naar een APK.



Android onderscheidt hierbij twee categorieën permissies. De eerste categorie omvat permissies die uitsluitend in het manifestbestand van de applicatie moeten worden opgenomen. Voor deze permissies is geen expliciete toestemming van de gebruiker vereist tijdens runtime. De tweede categorie omvat permissies die niet alleen in het manifestbestand moeten worden opgenomen, maar ook expliciet tijdens runtime door de gebruiker moeten worden goedgekeurd. Dit gebeurt via de functie `RequestPermission`, waarbij Android een dialoogvenster toont om de gebruiker om toestemming te vragen. Het is belangrijk dat de bijbehorende activiteit correct is geïnitialiseerd voordat deze aanvraag wordt gestart; anders beschouwt Android dit als een onveilige actie en zal de activiteit worden beëindigd.

Voor deze applicatie zijn de permissies als volgt:

- **Enkel manifest:** `ACCESS_BACKGROUND_LOCATION`, `FOREGROUND_SERVICE` en `FOREGROUND_SERVICE_LOCATION`
- **Met `RequestPermission`:** `ACCESS_COARSE_LOCATION` en `ACCESS_FINE_LOCATION`

### 3.1.2 Obfuscatie

Wanneer de Worker de callback van de `LocationManager` aanroept met het locatie-resultaat van de `FusedLocationProvider`, zijn er twee mogelijke paden:

- **Zonder obfuscator:** Indien er geen locatieobfuscator is ingesteld, fungeert de library als een eenvoudige facade. Als er een callbackfunctie is opgegeven, worden de coördinaten en het tijdstip als een `Triple` doorgegeven aan deze callback.
- **Met obfuscator:** Indien er wel een locatieobfuscator is ingesteld, wordt het resultaat van de `FusedLocationProvider` eerst naar de obfuscator gestuurd. Deze retourneert een geobfusceerde `Triple` met coördinaten en tijdstip, of `null` indien geen resultaat beschikbaar is. Indien een debugcallback is ingesteld, ontvangt deze zowel de originele als de geobfusceerde locatie. Alleen als er een geldig resultaat én een callbackfunctie is, wordt de callback aangeroepen met de geobfusceerde locatie.

```
fun useCallback(location: Location) {
    if (_locationObfuscator == null) {
        _callback?.invoke(Triple(location.latitude, location.longitude, location.time))
    } else {
        val obfuscatedLocation: LatLonTs? =
            this._locationObfuscator!!.obfuscateLocation(location)
        _debugCallback?.invoke(
            Triple(
                location.latitude,
                location.longitude,
```

```

        location.time
    ), obfuscatedLocation
)
// no callback if no result
obfuscatedLocation?.let { x -> _callback?.invoke(x) }
}
}

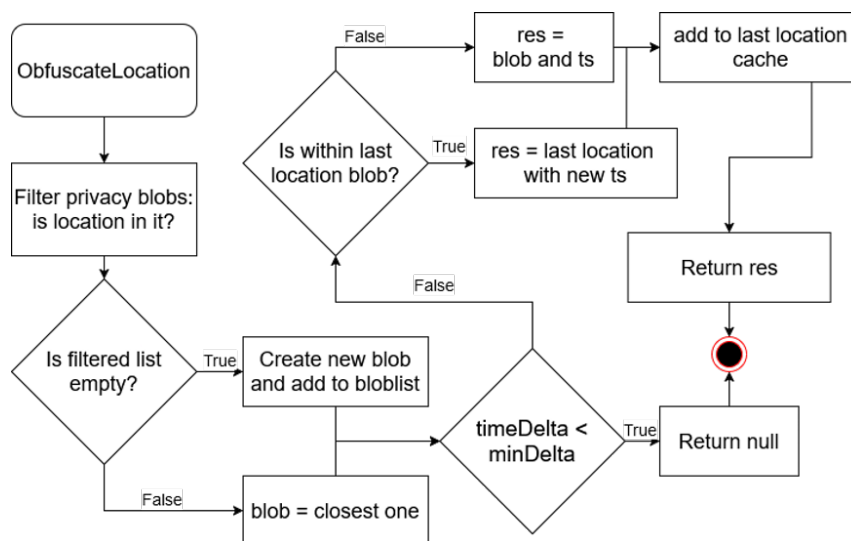
```

### 3.1.2.1 Standaard obfuscatiealgoritme

De library vereist een standaardimplementatie voor een obfuscatieklasse. Hiervoor wordt verwezen naar het werk van Toon Dehaene, waarin het concept van privacyzones centraal staat. Het toevoegen van enkel ruis aan locatiegegevens kan immers leiden tot het risico dat een aanvaller door middel van gemiddelden alsnog gevoelige informatie kan achterhalen. Het algoritme, zoals weergegeven in Figuur 3.2, illustreert de werking van deze aanpak.

- Het proces start met het filteren van de bestaande privacycirkels om te bepalen of de nieuwe locatie binnen een van deze cirkels valt. Tijdens dit proces wordt tevens de privacycirkel bijgehouden waarvan het middelpunt het dichtst bij de huidige locatie ligt.
- Indien de huidige locatie niet binnen een van de bestaande privacycirkels valt, wordt er een nieuwe cirkel aangemaakt. Het middelpunt van deze cirkel wordt bepaald door vanuit de huidige locatie, een willekeurige hoek en een willekeurige afstand (tussen 0 en de vooraf ingestelde maximale straal) te kiezen. Deze maximale straal is configureerbaar.
- Vervolgens wordt gecontroleerd of de vorige locatie voldoende lang geleden is verzonden. Indien dit niet het geval is, retourneert de obfuscator een `null`-waarde.
- Om jitter te voorkomen, wordt nagegaan of de huidige locatie zich in meerdere privacycirkels bevindt én binnen de laatst gebruikte privacycirkel valt. In dat geval wordt deze laatst gebruikte cirkel opnieuw geselecteerd, ongeacht of een andere cirkel dichterbij ligt. In alle andere gevallen wordt het middelpunt van de dichtstbijzijnde privacycirkel samen met de huidige tijdstempel als resultaat gebruikt.

De veranderingen tegenover het originele algoritme is als eerste dat er gecheckt wordt of de gebruiker in een privacycirkel bevindt voor het controleert of het lang genoeg gelden is. Daarnaast is een belangrijke toevoeging het mechanisme om jitter te voorkomen. Jitter kan optreden wanneer een gebruiker zich in het overlappende gebied van twee privacycirkels bevindt en de werkelijke locatie door kleine variaties (ruis) telkens net iets dichterbij een andere cirkel ligt. Zonder extra maatregelen zou het algoritme hierdoor continu kunnen wisselen tussen de twee cirkels, waardoor de uiteindelijke geobfusceerde locatie steeds verspringt. Dit maakt het voor een aanvaller mogelijk om, door het analyseren van deze verspringende locaties, de werkelijke positie van de gebruiker te verfijnen tot het overlappende gebied van



**Figure 3.2:** Flow diagram van het obfuscatie algoritme

de cirkels. Door altijd de laatst gebruikte privacycirkel te behouden zolang de gebruiker zich daarin bevindt, wordt dit risico geminimaliseerd.

### 3.1.3 Opslag

Omdat veel obfuscatiealgoritmen permanente opslag nodig hebben moet iedere obfuscatieklasse ook een load en store functie implementeren. Dit heeft als argument de `filesDir` van de hoofdactiviteit. Dit is een object waarmee de developer de opslag van de applicatie kan beïnvloeden. In de standaardimplementatie wordt dit gebruikt om voorgaande privacy cirkels op te slaan in een voor andere applicaties afgesloten file. [11]

### 3.1.4 Netwerk

De library heeft geen functionaliteit voor de locatie door te sturen naar een API endpoint. Dit moet geïmplementeerd worden in de callback function die de developer meegeeft aan de `LocationManager`. In de testapplicatie wordt de library, aanbevolen door Android zelf, `okhttp` gebruikt om de locatie door te sturen.

## 3.2 Applicatie

De testapplicatie is ontworpen om de werking van de library te demonstreren en te testen, zoals geïllustreerd in Figuur 3.3. De gebruikersinterface bestaat uit verschillende instellingen en knoppen die direct gekoppeld zijn aan de functionaliteit van de `LocationManager` en de locatieobfuscator.

De eerste vier instellingen betreffen de configuratie van de locatieverzameling:

- **Priority:** Bepaalt de nauwkeurigheid en het energieverbruik van de `FusedLocationProvider`.
- **Foreground:** Geeft aan of de locatieverzameling als foreground service (met notificatie) of als background task wordt uitgevoerd.
- **Interval (ms):** Stelt het tijdsinterval in tussen opeenvolgende locatieverzoeken aan de `FusedLocationProvider`. Dit interval heeft geen invloed op het interval van de standaard locatieobfuscator.
- **Max runtime (min):** Bepaalt de maximale duur van de background service. Volgens de beperkingen van Android kan deze maximaal 10 minuten per 15 minuten actief zijn.

Daarnaast zijn er twee instellingen specifiek voor de standaardimplementatie van de locatieobfuscator:

- **Blob radius:** De maximale straal van een privacycirkel.
- **Time between privacy:** De minimale tijd tussen het versturen van opeenvolgende locaties.

Onder de instellingen bevinden zich zes knoppen met de volgende functies:

- **Permissies aanvragen:** De eerste twee knoppen vragen de benodigde permissies op bij de gebruiker. De status van deze permissies wordt weergegeven en geüpdatet via LiveData zoals te zien in de tweede screenshot.
- **Clear obfuscator storage:** Hiermee wordt de permanente opslag van de obfuscator gewist, zodat testen telkens met een schone lei kunnen starten.
- **Nieuwe UUID genereren:** Hiermee wordt een unieke identificatie gegenereerd voor elke test, zodat resultaten eenvoudig te onderscheiden zijn.
- **Tracking starten/stoppen:** De laatste twee knoppen zijn voor locatieverzameling. De status van de tracking wordt eveneens via LiveData weergegeven in de linker knop, zodat de interface altijd de actuele status toont.

In het onderste tekstvak worden de resultaten gelogd die via de callback (niet de debugcallback) worden doorgegeven. Dit biedt tijdens het testen een directe indicatie dat de applicatie actief locaties verwerkt.

De drie screenshots tonen respectievelijk:

- De initiële opstart van de applicatie zonder verleende permissies.
- De situatie waarin de permissieknoppen de status van verleende permissies weergeven.
- De applicatie tijdens actieve tracking, waarbij een locatie is gelogd in het tekstvak.

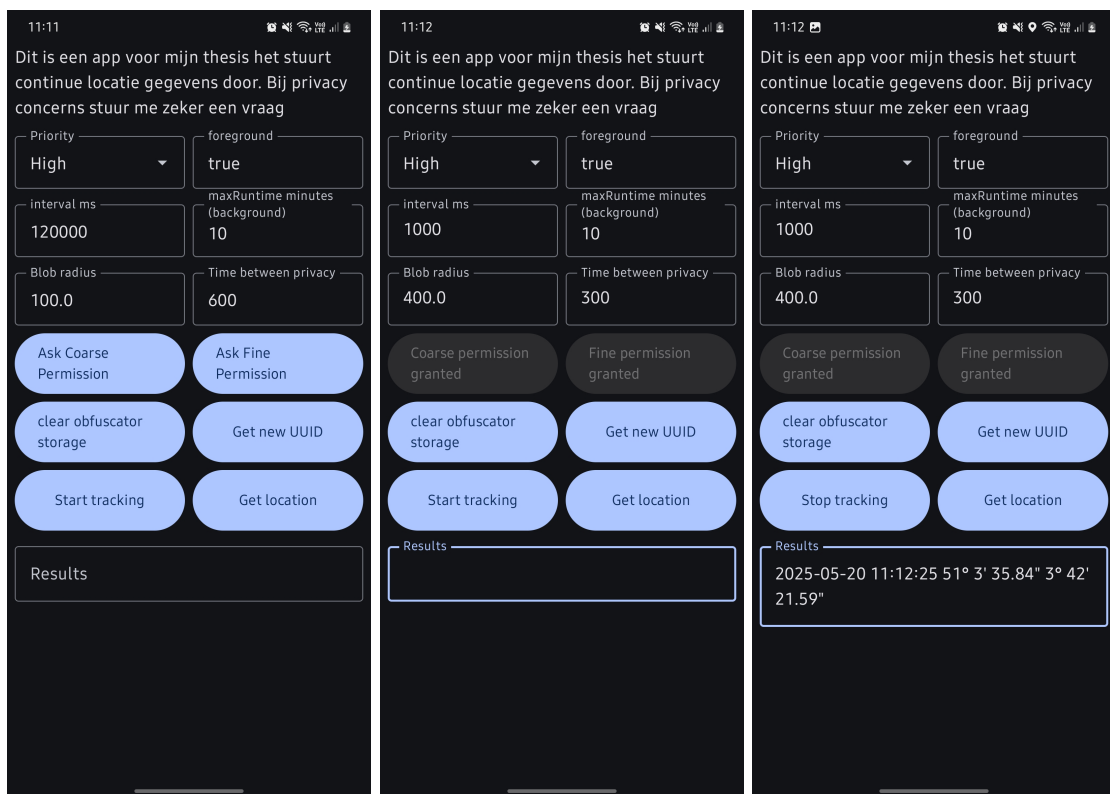


Figure 3.3: Visualisatie van de test applicatie

## 3.3 Dataverwerking

### 3.3.1 Verzameling

Voor de verzameling van data werd gebruik gemaakt van een Azure-webserver, specifiek opgezet voor studenten, met twee endpoints: `location` en `blobs`.

De `location`-endpoint ontving zowel de geobfusceerde als de originele locatiegegevens, afkomstig van de debugcallback, samen met relevante configuratieparameters. Deze gegevens werden opgeslagen in een CSV-bestand voor latere analyse.

De `blobs`-endpoint werd gebruikt om de privacycirkels door te sturen.

Om de afhankelijkheid van een continue internetverbinding te minimaliseren, werd een mechanisme geïmplementeerd waarbij de data lokaal werd opgeslagen wanneer er geen netwerkverbinding beschikbaar was. Zodra de verbinding werd hersteld, werden de opgeslagen gegevens alsnog doorgestuurd naar de server.

### 3.3.2 Verwerking

Voor de verwerking van de resultaten werden jupyter notebooks gehanteerd samen met de library Folium om de resultaten visueel te kunnen representeren. In elke notebook is er een

functie die de data inleest en locatiepunten die in de buurt van mijn privé zijn weg filtert. Dit zorgt ervoor dat alle gevisualiseerde data gepubliceerd kan worden. De eerste visualisatie was het verschil tussen coarse en fine Permissies vergelijken voor een experiment. Dit was beide op balanced power accuracy gebeurd zodat beide geen gps satellieten gebruikt. Het resultaat is te zien in het hoofdstuk experimenten in state of the art **??**. Een tweede test betrof het gebruik van twee telefoons met coarse permissies en twee met fine permissies. Voor elk paar werd één telefoon ingesteld op high accuracy en de andere op balanced power accuracy. Deze test werd echter niet opgenomen in het experimentenonderdeel van het hoofdstuk *State of the Art*, omdat er te veel externe factoren waren die de resultaten beïnvloedden, zoals bv. de beperkingen opgelegd door de energiebesparingsfunctionaliteiten van Android. De overige tests richtten zich op het evalueren van de standaardimplementatie van het obfuscatiealgoritme. Hierbij werden verschillende variabelen gevarieerd om de werking van het algoritme te analyseren en mogelijke verbeterpunten te identificeren. De inzichten die uit deze tests voortkwamen, werden gebruikt om de library verder te debuggen en te optimaliseren. Een uitgebreide bespreking van deze resultaten is te vinden in het hoofdstuk *Resultaten*.

### 3.4 Conclusie

In dit hoofdstuk werd de implementatie van de Android-library uitgebreid besproken. De library is ontworpen om locatiegegevens op een privacyvriendelijke manier te verzamelen en te verwerken. Het centrale beheer van locatieverzameling wordt uitgevoerd door de `LocationManager`, terwijl de `WorkManager` zorgt voor een flexibele en betrouwbare uitvoering van zowel foreground services als periodieke background tasks.

De library integreert een standaard obfuscatiealgoritme gebaseerd op privacycirkels, waarmee gevoelige informatie effectief wordt beschermd. Ontwikkelaars hebben de mogelijkheid om aangepaste obfuscatieklassen te implementeren en gebruik te maken van debugcallbacks voor analyse. Permanente opslag wordt ondersteund om privacycirkels en andere gegevens veilig te bewaren.

Daarnaast werd een testapplicatie ontwikkeld om data te verzamelen via een Azure-webserver. Hierbij werden mechanismen geïmplementeerd om gegevens lokaal op te slaan bij netwerkuitval, wat de betrouwbaarheid van de oplossing verhoogt. Deze aanpak minimaliseert de afhankelijkheid van een continue internetverbinding.

# 4

## Experimenten en resultaten

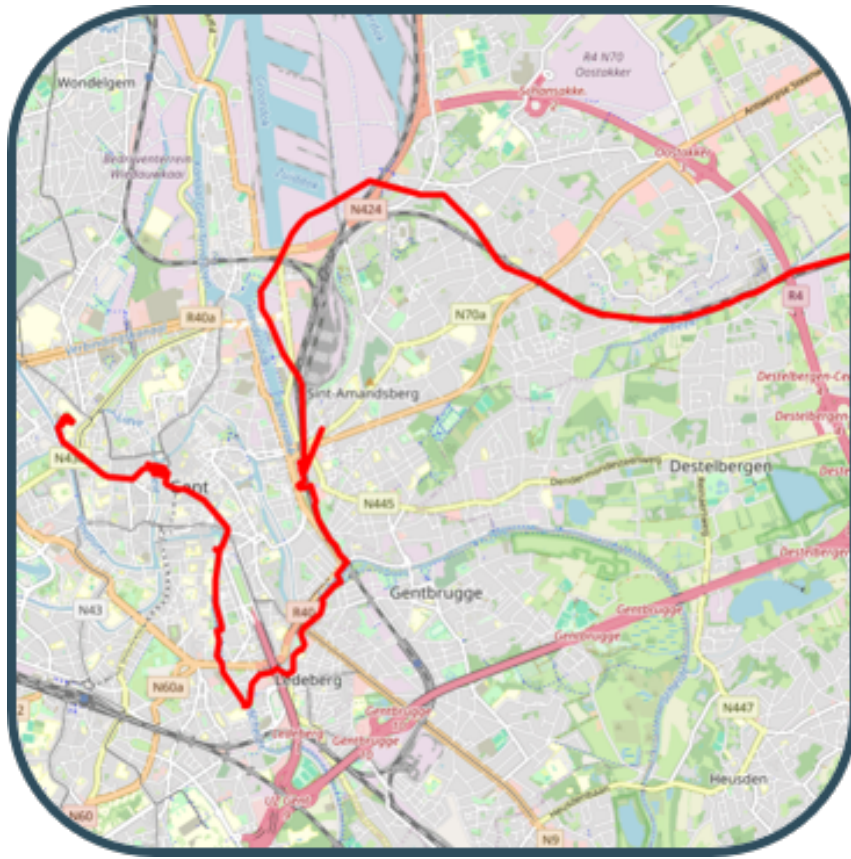
Naast de resultaten van de testen in verband met de werking van Android besproken in de state of the art. Gaat dit hoofdstuk over de uiteindelijke resultaten waarover de thesis gaat.

### 4.1 Visuele representatie

Voor het testen van de uiteindelijke library hebben we een specifieke route uitgestippeld die verschillende situaties omvat om de obfuscatie op diverse manieren te evalueren. De gevolgde route startte aan de Technologicampus Gent, waarna er naar de Korenmarkt werd gefietst. Daar werd gedurende twee uur op een terras gezeten om langdurig verblijf op één locatie te simuleren. Vervolgens werd er met de fiets naar de bibliotheek De Krook gereden, waar een korte stop werd ingelast om een kortstondig verblijf te testen. Daarna werd via een omweg naar station Dampoort gefietst, om vervolgens met de trein naar Lokeren te reizen en zo snel vervoer in een afgesloten ruimte te evalueren. Het resultaat van zo een trace is te zien in figuur 4.1.

In deze trace vallen enkele zaken op. Op locaties waar langere stops werden gemaakt, zoals op de campus, de Korenmarkt, De Krook en station Dampoort, ontstaat een cirkelvormig patroon door kleine variaties in de gemeten locatie. Bij het station is zelfs een uitschieter zichtbaar, vermoedelijk veroorzaakt door beweging onder het spoor waar het GPS-signaal minder duidelijk is.

Verder is het opvallend dat het gefietste traject zeer accuraat wordt weergegeven, terwijl het deel van de route per trein sterk afwijkt van het werkelijke traject. Dit kan te wijten zijn aan het kooi-van-Faraday-effect van de trein of aan de hogere snelheid waarmee wordt gereisd.



**Figure 4.1:** Echte pad

#### 4.1.1 Variabelen in kaart brengen

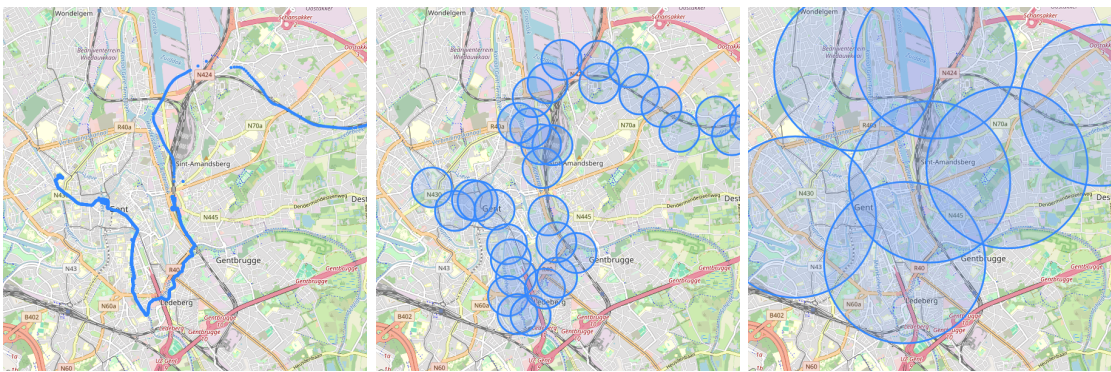
Dan wouden we de 2 parameters van het standaard obfuscatiealgoritme testen. Namelijk de radius van de privacycirkels en de tijdsinterval tussen verzonden punten. De variabelen werden samen met Toon Dehaene en Michiel Willocx zorgvuldig gekozen.

- **Radius(m):** 0, 50, 100, 200, 400, 800, 1600
- **Tijdsinterval(s):** 0, 60, 300, 600, 1800, 3600

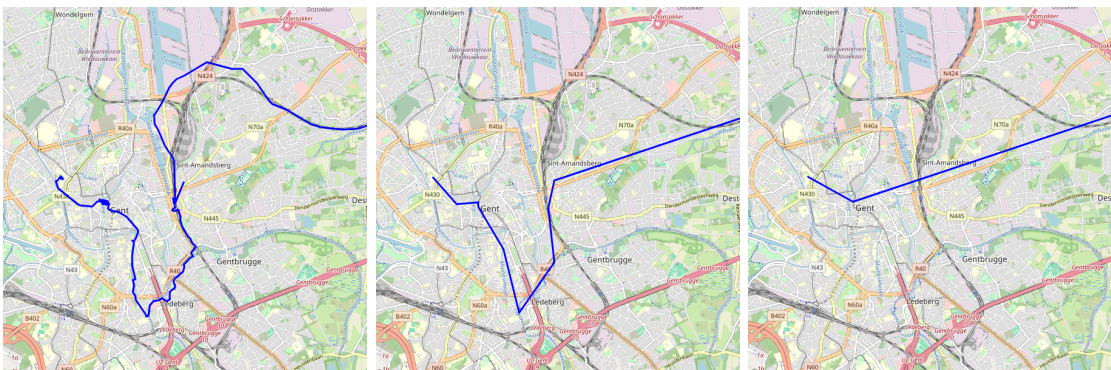
Deze variabelen resulteren in 42 mogelijke combinaties. De beschreven route werd daarom 11 keer afgelegd met 4 verschillende telefoons. In Figuur 4.2 wordt het effect van een privacyradius van 0, 400 en 1600 meter geïllustreerd. Omdat het tijdsinterval geen invloed heeft op het genereren van nieuwe privacycirkels, kan deze parameter afzonderlijk worden geëvalueerd. In de afbeeldingen zien we dat een radius van 0 meter de originele route exact volgt. Bij een privacyradius van 400 meter kunnen we aanzienlijke overlap tussen bepaalde cirkels zien, wat het gevolg is van de willekeurige plaatsing van nieuwe cirkels. Bij een radius van 1600 meter kunnen we zien dat slechts vier cirkels volstaan om bijna volledig klein Gent te bedekken.



In Figuur 4.3 wordt het effect van verschillende tijdsintervallen weergegeven. Deze afbeeldingen zijn gemaakt met een privacyradius van 0 meter, zodat enkel de invloed van het tijdsinterval zichtbaar is. De geobfusceerde locatie verschuift afhankelijk van waar de privacycirkels liggen. Uit de resultaten blijkt dat een tijdsinterval van 0 seconden ervoor zorgt dat het originele pad behouden blijft. Bij een tijdsinterval van 5 minuten is er meer detail zichtbaar rond de Korenmarkt, waar een langdurige stop plaatsvond. De rest van het pad is minder gedetailleerd, maar geeft nog steeds een algemeen beeld van de afgelegde route. Met een tijdsinterval van een uur zijn er slechts twee punten in Gent, waardoor er weinig nuttige informatie over het exacte traject beschikbaar is, behalve dat de campus en de Korenmarkt werden bezocht.



**Figure 4.2:** Vergelijking grootte privacycirkel

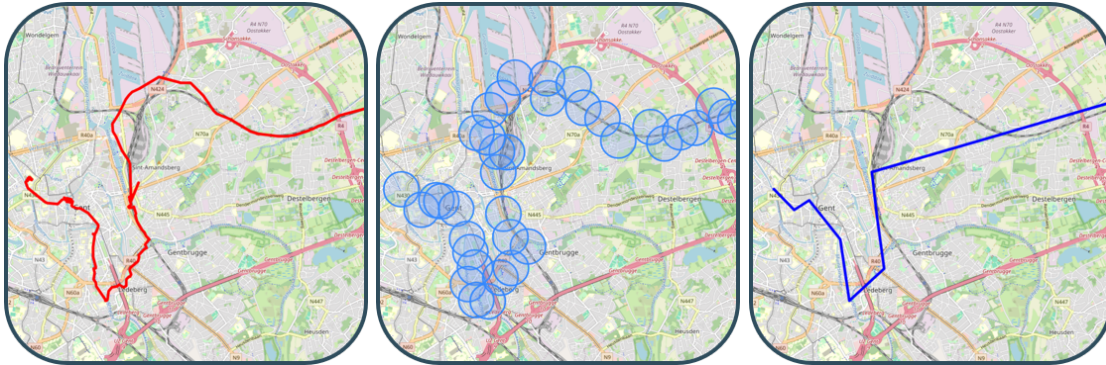


**Figure 4.3:** Vergelijking tijdsinterval locatiegegevens

#### 4.1.2 Uiteindelijke resultaat

Dan voor alles samen te testen hebben we 1 trace genomen die het meest accuraat was naar de echte beweging van de route. Deze hebben we dan met het algoritme dat lokaal loopt in de library voor de 42 variabelen combinaties laten berekenen. Al deze combinaties zijn te veel voor op deze thesis maar zijn te vinden op deze github page om zo overzichtelijker te kunnen vergelijken. De gekozen visualisatie is te zien in deze figuur 4.4. De gekozen variabelen voor

deze visualisatie is een privacycirkel met een straal van 400 meter en een tijds interval van 5 minuten.



**Figure 4.4:** Obfuscatiealgoritme gevisualiseerd

# 5

## Evaluatie en discussie

### 5.1 Evaluatie van het systeem

#### 5.1.1 Beperkingen

In de huidige versie van de library is slechts beperkt onderzoek verricht naar de langdurige stabiliteit van locatieverzameling in de achtergrond en naar mogelijke onbekende beperkingen die verder reiken dan de maximale threaddtijd van 10 minuten per 15 minuten. Om dit grondig te testen, zouden meerdere langdurige runtimes moeten worden voorzien.

Voor de testapplicatie is een fallbackmechanisme geïmplementeerd voor mislukte HTTP-verzoeken, waarbij deze tijdelijk worden opgeslagen in de shared preferences van Android. Echter, shared preferences zijn niet geschikt voor het verwerken van grote hoeveelheden data, wat leidt tot vertragingen bij het lezen en schrijven, vooral wanneer er gedurende langere tijd geen netwerkverbinding beschikbaar is. Bovendien kan de plotselinge piek in internetverkeer bij het herstellen van een netwerkverbinding resulteren in crashes op Android. Verdere uitbreiding van de testapplicatie vereist daarom throttlingmechanismen en een robuustere permanente opslagoplossing, met name voor scenario's waarin testapparaten langdurig geen internetverbinding hebben.

## 5.2 Verder werk

## 5.3 Gebruik van de library

### 5.3.1 Initialisatie

Het gebruik van de library verloopt in enkele duidelijke stappen. Eerst wordt een instantie van de `LocationManager` aangemaakt, waarbij de `MainActivity` als parameter wordt meegegeven. Dit is noodzakelijk om een observer te kunnen koppelen aan de lifecycle van de `MainActivity`. Vervolgens wordt een `LocationObfuscator` ingesteld; dit kan zowel de standaardimplementatie zijn als een door de ontwikkelaar zelf geïmplementeerde klasse. Ten slotte wordt een callbackfunctie geregistreerd, die door de library wordt aangeroepen zodra er een nieuwe locatie beschikbaar is.

```
class MainActivity : ComponentActivity() {
    private val locationManager = LocationManager.getInstance(this)
    init {
        locationManager.locationObfuscator = LocationObfuscatorV1.getInstance()
        locationManager.callback =
            { location -> sendLocationToServer(location) }
    }
}
```

### 5.3.2 LocationObfuscator zelf implementeren

Om een eigen locatie-obfuscator te implementeren, kan men onderstaande aanpak volgen. De eenvoudigste implementatie bestaat erin dat de locatie verkregen via de `FusedLocationProvider` als argument wordt doorgegeven aan de `obfuscateLocation`-functie, die vervolgens een tuple van coördinaten en tijdstempel retourneert. De overige drie functies (`load`, `store` en `clearStorage`) zijn bedoeld voor het beheren van permanente opslag. Indien nodig kunnen variabelen via de constructor worden meegegeven. Het `filesDir`-object dat aan deze functies wordt doorgegeven, biedt toegang tot de opslagruimte van de applicatie.

```
import android.location.Location
import be.kuleuven.milanschollier.safegps.LatLonTs
import be.kuleuven.milanschollier.safegps.LocationObfuscator
import java.io.File
class ImplementationExample private constructor() : LocationObfuscator {
    override fun obfuscateLocation(location: Location): LatLonTs {
        return LatLonTs(location.latitude, location.longitude, location.time)
    }
    override fun load(filesDir: File) {return}
```

```
    override fun store(filesDir: File) {return}  
    override fun clearStorage(filesDir: File) {return}  
}
```

### 5.3.3 Live data

De developer kan gebruik maken van live data in de library. Zo is `finePermission`, `coarsePermission` en `running live data` dat de applicatie intuïtief kan laten reageren op de werking van de library.

```
val finePermission = locationManager.finePermission.observeAsState()  
Button(  
    onClick = {  
        locationManager.getFinePermission(true)  
    },  
    modifier = Modifier  
        .weight(1f)  
        .fillMaxSize(),  
    enabled = finePermission.value == false  
) {  
    Text(text = if (finePermission.value == true) "Fine permission granted" else "Ask F  
}
```

## 5.4 Conclusie



# 6

## Conclusie

Deze thesis demonstreert dat het mogelijk is om locatiegegevens on-device te obfusceren zonder de bruikbaarheid voor analyse significant aan te tasten. Door het ontwikkelen van een Android-bibliotheek die locaties en tijdstempels vervormt via ruis en privacycirkels, wordt voorkomen dat individuele gebruikers geïdentificeerd kunnen worden via hun bewegingspatroon. De kern van de oplossing ligt in het behoud van data-utility: geaggregeerde inzichten, zoals mobiliteitstrends of drukteanalyses, blijven mogelijk, terwijl gevoelige individuele informatie wordt beschermd.

Een belangrijk succes is de volledige onafhankelijkheid van externe servers. Alle verwerking vindt plaats op het apparaat zelf, waardoor de originele, nauwkeurige locatie nooit het toestel verlaat. Dit elimineert het risico op datalekken bij overdracht of opslag. Technisch werd dit gerealiseerd door een modulaire ontwerp, waarbij de bibliotheek naadloos integreert met Android's Fused Location Provider en WorkManager voor betrouwbare achtergrondverwerking. Daarnaast biedt de implementatie van een standaard obfuscatiealgoritme op basis van privacycirkels een evenwicht tussen anonymisering en nauwkeurigheid, geïnspireerd op eerder onderzoek.

De bibliotheek verlaagt niet alleen de implementatiedrempel voor ontwikkelaars door complexe privacytechnieken te abstraheren, maar draagt ook bij aan een paradigmaverschuiving in databeheer. In plaats van privacy als een afterthought te behandelen, wordt bescherming een inherent onderdeel van het ontwerp. Dit sluit aan bij groeiende regelgevende eisen en maatschappelijke vraag naar transparante datapraktijken.

Hoewel verdere optimalisatie mogelijk blijft, toont deze aanpak aan dat technische oplossingen een sleutelrol kunnen spelen in het verzoenen van data-innovatie en individuele rechten.

Door privacy-by-design toegankelijk te maken, zet deze thesis een stap richting een toekomst waarin technologie niet ten koste gaat van gebruikerscontrole, maar deze net versterkt.



# Bibliography

- [1] A. Stiru, "Hack bij locatiedatabroker gravity analytics treft miljoenen smartphonegebruikers", 2025. [Online]. Available: <https://tweakers.net/nieuws/230770/hack-bij-locatiedatabroker-gravity-analytics-treft-miljoenen-smartphonegebruikers.html>.
- [2] A. Bannister, 2021. [Online]. Available: <https://portswigger.net/daily-swig/trilateration-vulnerability-in-dating-app-bumble-leaked-users-exact-location>.
- [3] C. Newton, 2019. [Online]. Available: <https://www.theverge.com/interface/2019/3/28/18285274/grindr-national-security-cfius-china-kunlun-military>.
- [4] [Online]. Available: <https://developer.android.com/guide/components/activities/activity-lifecycle>.
- [5] [Online]. Available: <https://developer.android.com/develop/background-work/background-tasks#user-initiated>.
- [6] [Online]. Available: <https://developer.android.com/develop/background-work/services/alarms/schedule>.
- [7] [Online]. Available: <https://developer.android.com/develop/background-work/background-tasks/persistent>.
- [8] [Online]. Available: <https://developer.android.com/kotlin/coroutines>.
- [9] [Online]. Available: <https://developer.android.com/develop/sensors-and-location/location/battery>.
- [10] [Online]. Available: <https://refactoring.guru/design-patterns/singleton>.
- [11] [Online]. Available: <https://developer.android.com/training/data-storage/app-specific>.





## Code van het obfuscatie algoritme

```
override fun obfuscateLocation(location: Location): LatLonTs? {
    val lat = location.latitude
    val lon = location.longitude
    val ts: Long = location.time

    val matchedReports = historyBlobs.filter { report ->
        Geodesic.WGS84.Inverse(
            report.first,
            report.second,
            lat,
            lon,
            GeodesicMask.DISTANCE
        ).s12 < report.third
    }

    val blob: PrivacyBlob
    if (matchedReports.isEmpty()) {
        val sampledRadius = radiusSampler()
        val sampledAngle = Math.toDegrees(angleSampler())
        val res = Geodesic.WGS84.Direct(lat, lon, sampledAngle, sampledRadius)
        blob = PrivacyBlob(res.lat2, res.lon2, _blobRadius)
        println("Generated new privacy blob: $blob")
        historyBlobs.add(blob)
    }
}
```

```

    } else {
        blob = matchedReports.minBy { report ->
            Geodesic.WGS84.Inverse(
                report.first,
                report.second,
                lat,
                lon,
                GeodesicMask.DISTANCE
            ).s12
        }
    }
}

val lastLocation = perturbedLocationCache.lastOrNull() ?: LatLonTs(0.0, 0.0, 0)

if (ts - lastLocation.third < _deltaTime * 1000) {
    return null
}
//Voor jitter tegen te gaan als in de buurt van vorige locatie die
//privacy blob doorsturen ongeacht andere dichters is
val res: LatLonTs = if (Geodesic.WGS84.Inverse(
    lastLocation.first,
    lastLocation.second,
    lat,
    lon,
    GeodesicMask.DISTANCE
).s12 < _blobRadius
) {
    LatLonTs(lastLocation.first, lastLocation.second, ts)
} else {
    LatLonTs(blob.first, blob.second, ts)
}

perturbedLocationCache.add(res)
if (perturbedLocationCache.size > 5) perturbedLocationCache.removeAt(0)
return res
}

```



## Beschrijving van deze masterproef in de vorm van een wetenschappelijk artikel

The thesis should also contain a short scientific article. If you write your thesis in Dutch, you must write the article in English, and vice versa. We advise you to employ the IEEE Manuscript Templates for Conference Proceedings (<https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhnncsfqn>). Compile the article in another project and include the generated pdf file as shown below:

# Conference Paper Title\*

\*Note: Sub-titles are not captured in Xplore and should not be used

1<sup>st</sup> Given Name Surname

*dept. name of organization (of Aff.)*

*name of organization (of Aff.)*

City, Country

email address or ORCID

2<sup>nd</sup> Given Name Surname

*dept. name of organization (of Aff.)*

*name of organization (of Aff.)*

City, Country

email address or ORCID

3<sup>rd</sup> Given Name Surname

*dept. name of organization (of Aff.)*

*name of organization (of Aff.)*

City, Country

email address or ORCID

4<sup>th</sup> Given Name Surname

*dept. name of organization (of Aff.)*

*name of organization (of Aff.)*

City, Country

email address or ORCID

5<sup>th</sup> Given Name Surname

*dept. name of organization (of Aff.)*

*name of organization (of Aff.)*

City, Country

email address or ORCID

6<sup>th</sup> Given Name Surname

*dept. name of organization (of Aff.)*

*name of organization (of Aff.)*

City, Country

email address or ORCID

**Abstract**—This document is a model and instructions for  $\LaTeX$ . This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. \*CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

**Index Terms**—component, formatting, style, styling, insert

## I. INTRODUCTION

This document is a model and instructions for  $\LaTeX$ . Please observe the conference page limits.

## II. EASE OF USE

### A. Maintaining the Integrity of the Specifications

The IEEEtran class file is used to format your paper and style the text. All margins, column widths, line spaces, and text fonts are prescribed; please do not alter them. You may note peculiarities. For example, the head margin measures proportionately more than is customary. This measurement and others are deliberate, using specifications that anticipate your paper as one part of the entire proceedings, and not as an independent document. Please do not revise any of the current designations.

## III. PREPARE YOUR PAPER BEFORE STYLING

Before you begin to format your paper, first write and save the content as a separate text file. Complete all content and organizational editing before formatting. Please note sections III-A–III-E below for more information on proofreading, spelling and grammar.

Keep your text and graphic files separate until after the text has been formatted and styled. Do not number text heads— $\LaTeX$  will do that for you.

Identify applicable funding agency here. If none, delete this.

### A. Abbreviations and Acronyms

Define abbreviations and acronyms the first time they are used in the text, even after they have been defined in the abstract. Abbreviations such as IEEE, SI, MKS, CGS, ac, dc, and rms do not have to be defined. Do not use abbreviations in the title or heads unless they are unavoidable.

### B. Units

- Use either SI (MKS) or CGS as primary units. (SI units are encouraged.) English units may be used as secondary units (in parentheses). An exception would be the use of English units as identifiers in trade, such as “3.5-inch disk drive”.
- Avoid combining SI and CGS units, such as current in amperes and magnetic field in oersteds. This often leads to confusion because equations do not balance dimensionally. If you must use mixed units, clearly state the units for each quantity that you use in an equation.
- Do not mix complete spellings and abbreviations of units: “Wb/m<sup>2</sup>” or “webers per square meter”, not “webers/m<sup>2</sup>”. Spell out units when they appear in text: “. . . a few henries”, not “. . . a few H”.
- Use a zero before decimal points: “0.25”, not “.25”. Use “cm<sup>3</sup>”, not “cc”.)

### C. Equations

Number equations consecutively. To make your equations more compact, you may use the solidus ( / ), the exp function, or appropriate exponents. Italicize Roman symbols for quantities and variables, but not Greek symbols. Use a long dash rather than a hyphen for a minus sign. Punctuate equations with commas or periods when they are part of a sentence, as in:

$$a + b = \gamma \quad (1)$$

Be sure that the symbols in your equation have been defined before or immediately following the equation. Use “(1)”, not

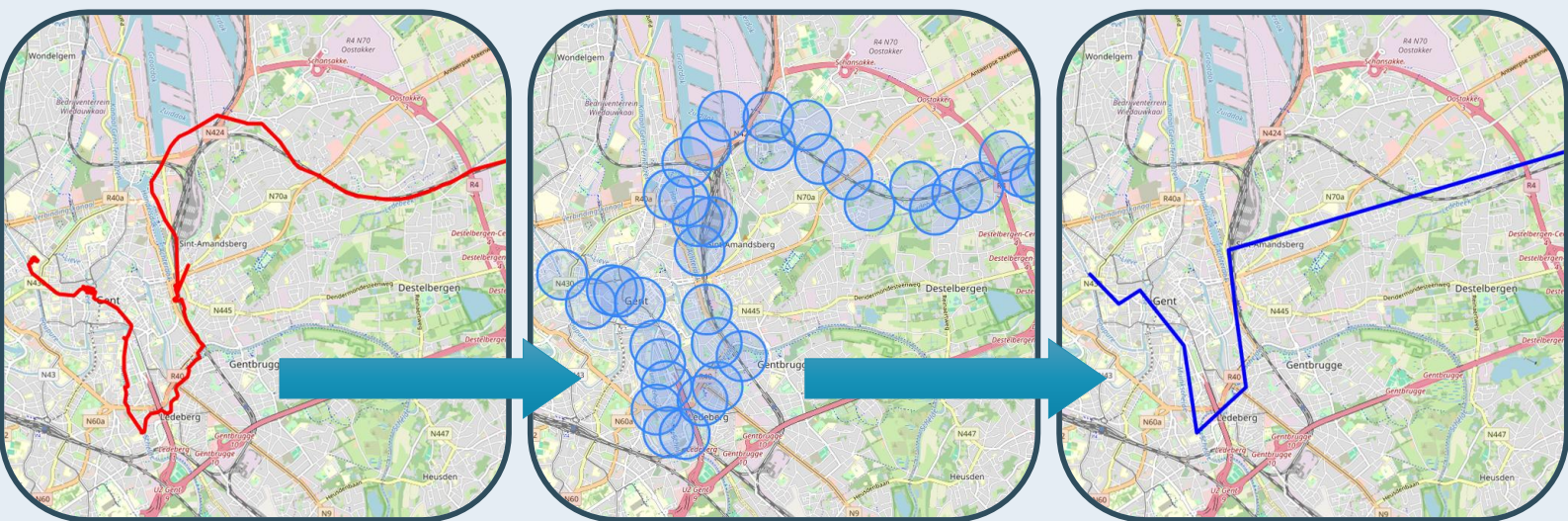
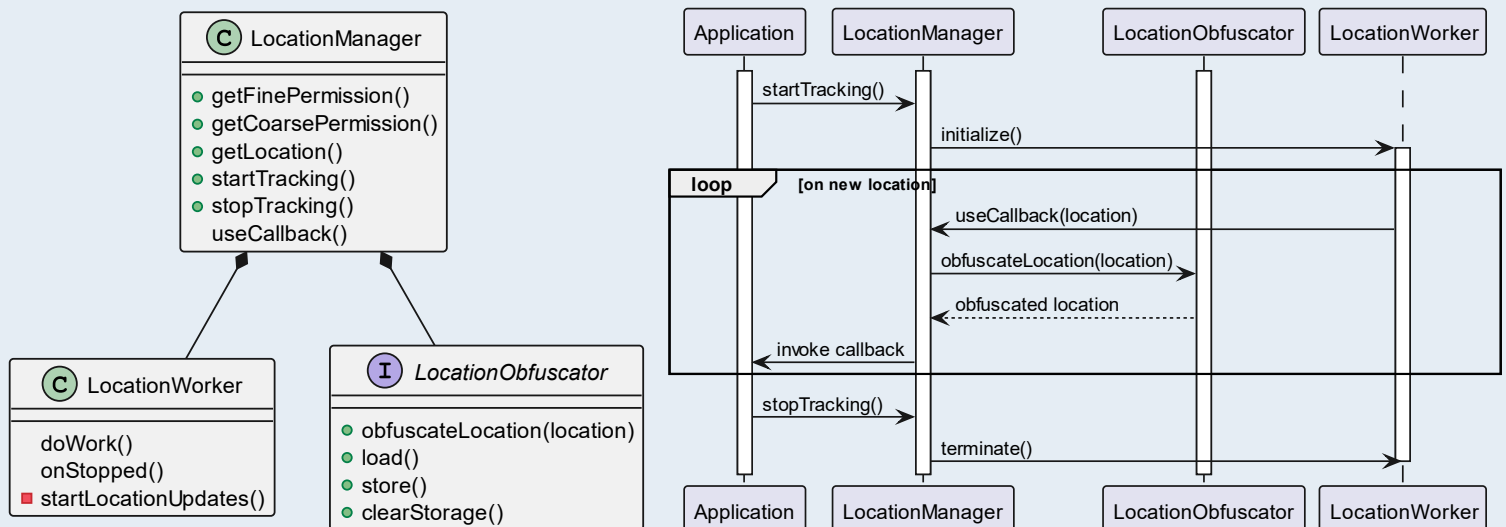
# C

Poster

# Ontwikkeling van een smartphone sdk voor het obfusceren van locatie data

## Abstract

Locatiedata biedt waardevolle inzichten, maar brengt aanzienlijke privacy risico's met zich mee, zoals het afleiden van gevoelige locaties en sociale grafen. Deze thesis introduceert een innovatieve Android-bibliotheek die locatiedata obfusceert door ruis toe te voegen aan locaties en tijdstempels. Dit voorkomt dat individuele sporen aan gebruikers worden gekoppeld, terwijl de bruikbaarheid van de data behouden blijft. De bibliotheek werkt volledig op het apparaat, zodat nauwkeurige gegevens nooit het toestel verlaten. Experimenten tonen de effectiviteit aan en vergelijken de oplossing met Android 12's Approximate Location-functie, wat bijdraagt aan verbeterde mobiele privacy.





**KU LEUVEN - GENT**

Faculteit Industriële Ingenieurswetenschappen  
Campus Rabot, Gebroeders De Smetstraat 1

9000 Gent, België

tel: + 32 9 265 86 10

[iiw.gent@kuleuven.be](mailto:iiw.gent@kuleuven.be)

[www.iw.kuleuven.be](http://www.iw.kuleuven.be)

